Artificial Intelligence
UNIT III
Adversarial Search and Game theory

Prepared By

Mrs. Priyadarshani Doke

Index

- Optimal Decisions in Games
- Heuristic Alpha–Beta Tree Search
- Monte Carlo Tree Search
- Stochastic Games
- Partially Observable Games
- Limitations of Game Search Algorithms
- Constraint Satisfaction Problems (CSP)
- Constraint Propagation: Inference in CSPs, Backtracking Search for CSPs
- **Case Study:**

1. **AlphaGo → Reinforcement Learning + MCTS**
2. **Libratus → Game Theory + Imperfect Information**
3. **Deep Blue → Adversarial Search + Minimax + Alpha-Beta Pruning**

Games: hard topic

1. Games are a big deal in AI
2. Games are interesting to AI because they are too hard to solve
3. Chess has a branching factor of 35, with 35^{100} nodes ↑ 10^{154}
4. Need to make some decision even when the optimal decision is infeasible

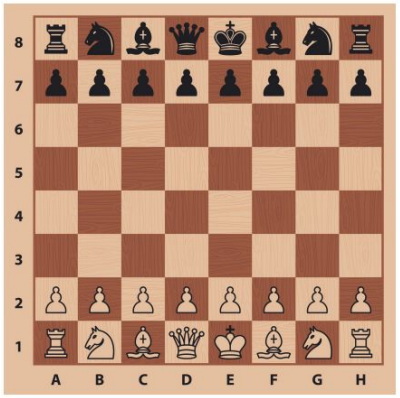
Adversarial Search

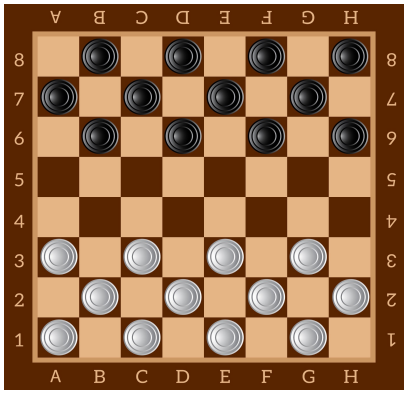
- Adversarial search in AI is the family of algorithms used when agents compete against each other, most commonly in two-player zero-sum games like **chess, tic-tac-toe, or Go**.
- The core methods are the ***Minimax algorithm*** and its optimization, ***Alpha-Beta pruning***, which allow an AI to anticipate an opponent's moves and choose the best counter-strategy.
- The environment with more than one agent is termed as multi-agent environment, in which **each agent is an opponent** of other agent and playing against each other. Each agent needs to consider the action of other agent and effect of that action on their performance.
- Searches in which two or more players with conflicting goals are trying to **explore the same search space for the**

What Is Adversarial Search?

- Adversarial search problems =games.

- **Definition**: A search strategy for competitive environments where one agent's gain is another's loss.
- **Goal**: Find the optimal move by considering both the agent's actions and the opponent's possible responses.
- **Applications**: Board games (chess, Go, checkers), card games (poker), and real-world competitive domains like trading, autonomous driving, and military simulations.

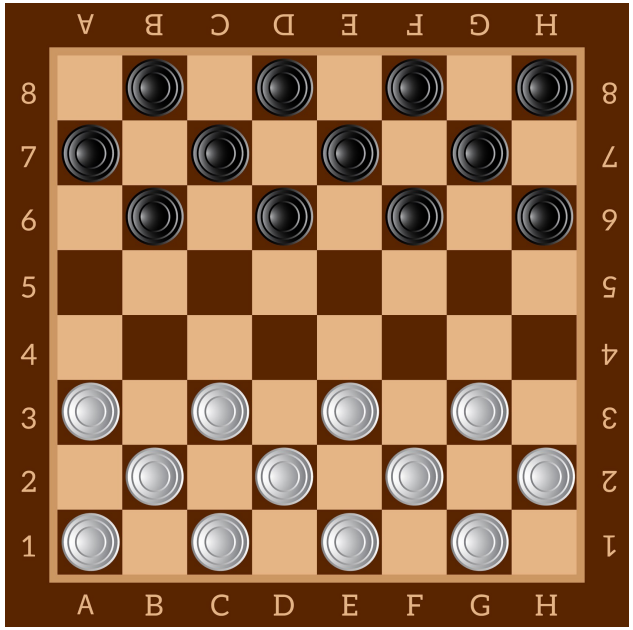




Adversarial Search

Checkers:

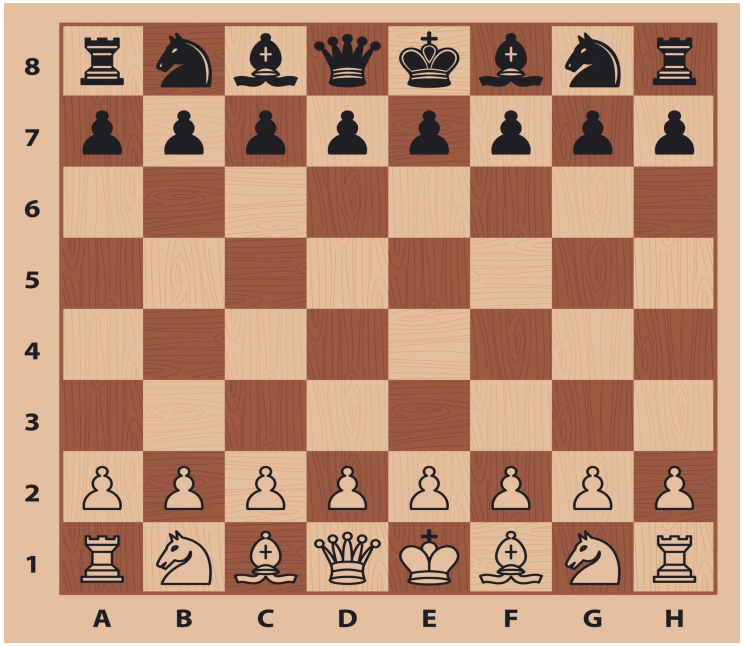
- Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994.
- Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 positions.



Adversarial Search

Chess:

- In 1949, Caude E. Shannon in his paper "Programming a Com- puter for Playing Chess", suggested *Chess* as an AI problem for the community.
- Deep Blue defeated human world champion Gary Kasparov in a six-game match in 1997.

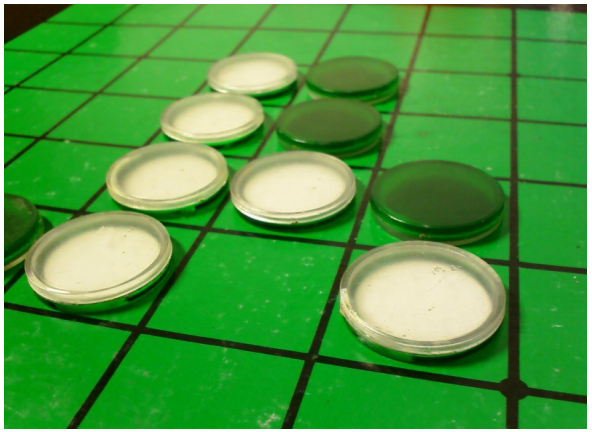


Adversarial Search

Go: $b > 300$! Google Deep mind Project AlphaGo. In 2016, AlphaGo beat both Fan Hui, the European Go champion and Lee Sedol the worlds best player.

Othello: Several computer othello exists and human champions refuse to compete against computers, that are too good.





By DonarreiskoerBy Paul 012 via Wikimedia Commons

Search Under Adversarial Circumstances

- In **Multiagent environments**, each agent needs to consider the actions of other agents and how they affect its own welfare.
- The unpredictability of these other agents can introduce **contingencies** into the agent's problem-solving process
- In **competitive** environments, the agents' goals are in conflict, giving rise GAME to **adversarial search** problems—often known as **games**.
- most common rather specialized kinds of games in **AI** are game deterministic
  - turn-taking
  - two-player
  - **zero-sum games** of **perfect information** (such as chess).

Search Under Adversarial Circumstances

- **Deterministic** means fully observable environments in which two agents act alternately and in which the utility values at the end of the game are always equal and opposite.
- For example, if one player wins a game of chess, the other player necessarily loses.
- It is this opposition between the agents' utility functions that makes the situation adversarial.

Search Under Adversarial Circumstances

- The state of a game is easy to represent, and agents are usually restricted to a small number of actions whose outcomes are defined by precise rules.

- Physical games, such cricket, football ,ice hockey, have much more complicated descriptions, a much larger range of possible actions, and rather imprecise rules defining the legality of actions.

Search Under Adversarial Circumstances

- For example, chess has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{100} nodes

- although the search graph has "only" about 1040 distinct nodes

- Games, like the real world, therefore require the ability to make some decision even when calculating the ***optimal*** **decision is infeasible**.

Search Under Adversarial Circumstances

- Game-playing programs developed by AI researchers since the beginning of the modern AI era (chess, checkers in 1950s)

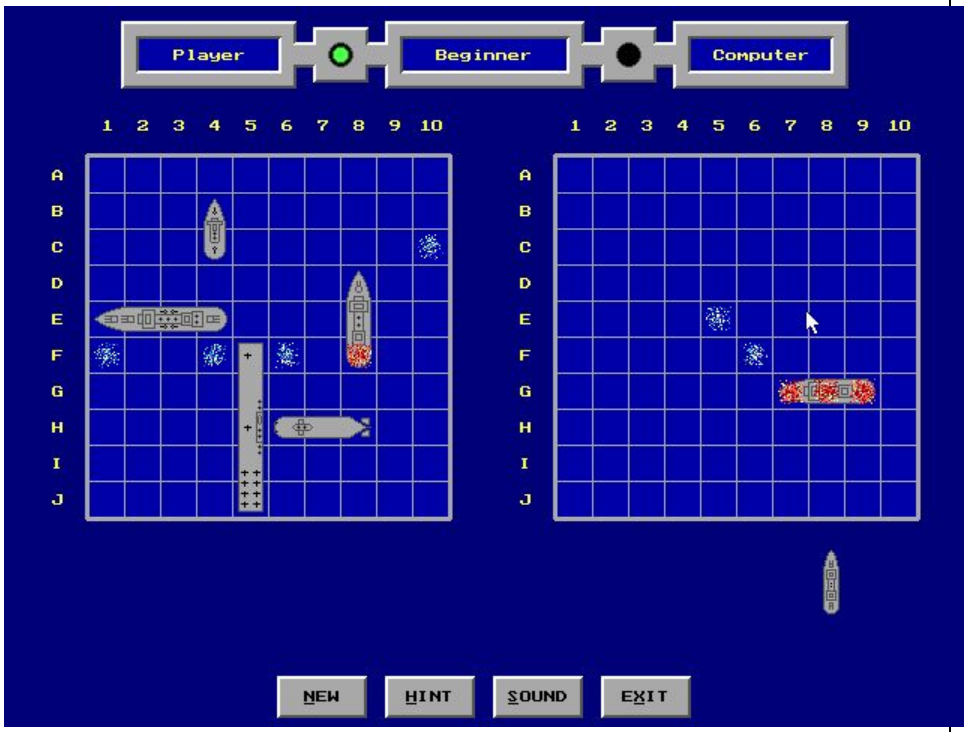
- **Game Search**
  - Sequences of player's decisions *we control*
  - Decision of other player(s) *we do not control*

- **Perfect information:** A game with the perfect information is that in which agents can look into the complete board. Agents have all the information about the game, and they can see each other moves also. Examples are Chess, Checkers, Go, etc.

- **Imperfect information:** If in a game agents do not have all information about the game and not aware with what's going on, **no complete search, search is complete earlier,** such type of games are called the game with imperfect information, such as Battleship, blind tic tac toe, Bridge, etc.

Battleship


BATTLESHIP
7+
2

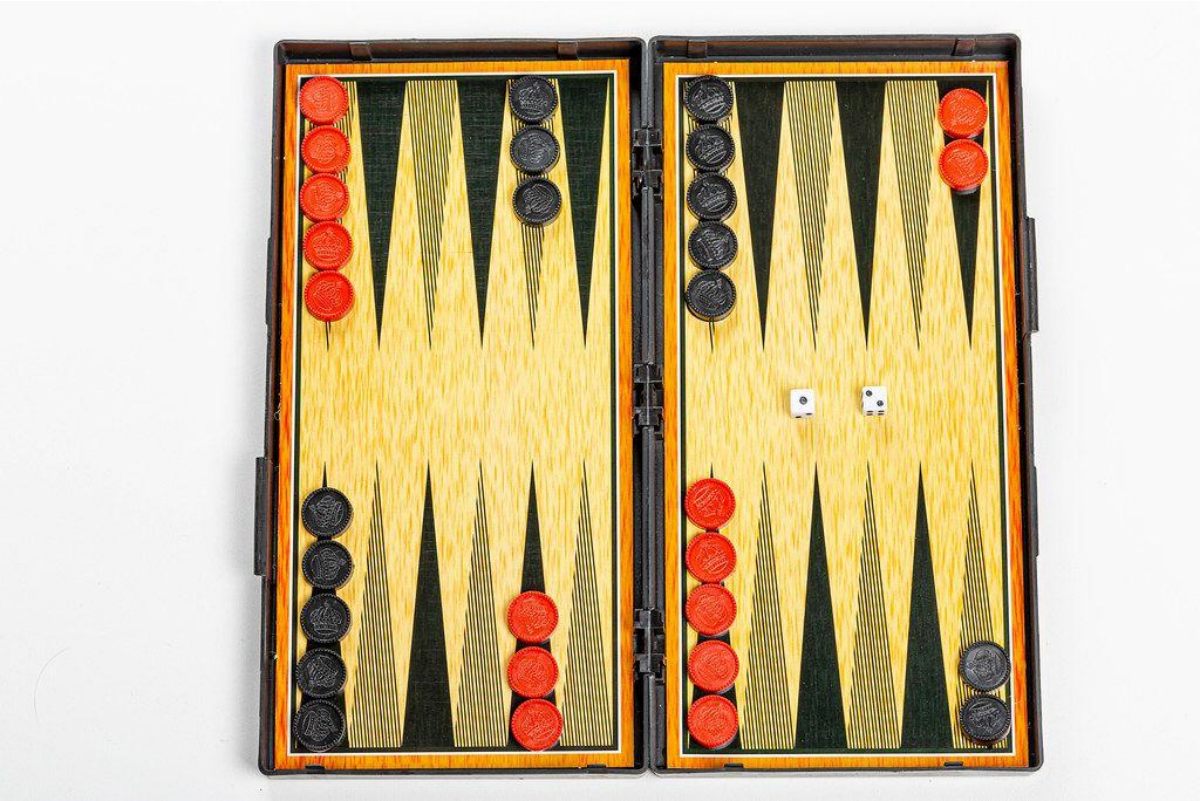

Player
Beginner
Computer
1 2 3 4 5 6 7 8 9 10
A B C D E F G H I J
NEW
HINT
SOUND
EXIT

blind tic tac toe



- **Deterministic games:** Deterministic games are those games which follow a strict pattern and set of rules for the games, and there is no randomness associated with them. Examples are chess, Checkers, Go, tic-tac-toe, etc.
- **Non-deterministic games:** Non-deterministic are those games which have various unpredictable events and has a factor of chance or luck. This factor of chance or luck is introduced by either dice or cards. These are random, and each action response is not fixed. Such games are also called as stochastic games.
- Example: Backgammon, Monopoly, Poker, etc.

Backgammon



The first player to bear off (remove) all 15 of their checkers from the board wins the game.

Chance/ Randomness

- **Chance** means that **some part of the game outcome is determined randomly**, rather than completely controlled by the player's decision.
- **Chance ≠ Imperfect Information**

  These are two different concepts.

  Chance:

  "I don't know what random outcome will occur."

  Example: **Ludo dice**
- **Imperfect Information:**

  "I cannot see or know the complete state of the game."

  Example: **Poker – I cannot see the opponent's cards.**

Types of games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble nuclear war

We are mostly interested in deterministic games, fully ob-servable environments, zero-sum, where two agents act al-ternately.

Identify whether the following games are deterministic or non-deterministic. Give a suitable reason for each.

- Backgammon
- Tic-Tac-Toe
- Poker
- Chess
- Ludo
- Othello (Reversi)
- Monopoly
- Go
- Snakes and Ladders
- Checkers
- Blackjack
- Connect Four
- Mancala
- Teen Patti
- Nine Men's
- Morris
- Chinese Checkers
- YahtzeeShogi
- Rummy
- Gomoku (Five-in-a-Row)
- Roulette
- Xiangqi (Chinese Chess)
- UNO
- Draughts
- Craps
- Tower of Hanoi (puzzle, not strictly a competitive game)

Game	Information	Type	AI Concept
Backgammon	Perfect	Non-deterministic	Expectiminimax
Tic-Tac-Toe	Perfect	Deterministic	Minimax
Poker	Imperfect	Non-deterministic	Game Theory
Chess	Perfect	Deterministic	Minimax + Alpha-Beta Pruning
Ludo	Perfect	Non-deterministic	Expectiminimax
Othello (Reversi)	Perfect	Deterministic	Minimax + Alpha-Beta
Monopoly	Perfect	Non-deterministic	Expectiminimax

Game	Information	Type	AI Concept
Go	Perfect	Deterministic	Monte Carlo Tree Search + Reinforcement Learning
Snakes and Ladders	Perfect	Non-deterministic	Probability + Expectiminimax
Checkers	Perfect	Deterministic	Minimax + Alpha-Beta Pruning
Blackjack	Imperfect	Non-deterministic	Probability + Game Theory
Connect Four	Perfect	Deterministic	Minimax + Alpha-Beta Pruning
Mancala	Perfect	Deterministic	Minimax + Search
Teen Patti	Imperfect	Non-deterministi	Probability +

Category	Examples
Perfect Information	Chess, Go, Checkers, Tic-Tac-Toe, Ludo, Backgammon
Imperfect Information	Poker, Blackjack, Rummy, Teen Patti, UNO
Deterministic Games	Chess, Go, Checkers, Tic-Tac-Toe, Othello, Shogi
Non-deterministic Games	Ludo, Backgammon, Monopoly, Poker, Blackjack, Yahtzee
Minimax	Chess, Checkers, Tic-Tac-Toe, Othello, Connect Four
Expectiminimax	Backgammon, Ludo, Monopoly
Game Theory	Poker, Blackjack, Rummy, Teen Patti
Probability	Roulette, Craps, Bingo, Yahtzee
State-Space Search	Tower of Hanoi
MCTS + Reinforcement Learning	Go / AlphaGo

Game Setup

- Consider games with two players, whom we call MAX and MIN
- **MAX moves first**, and then they take turns moving until the game is over.
- At the end of the game,
  - **points** are awarded to the **winning player** and
  - **penalties** are given to the **loser**.

Game Setup

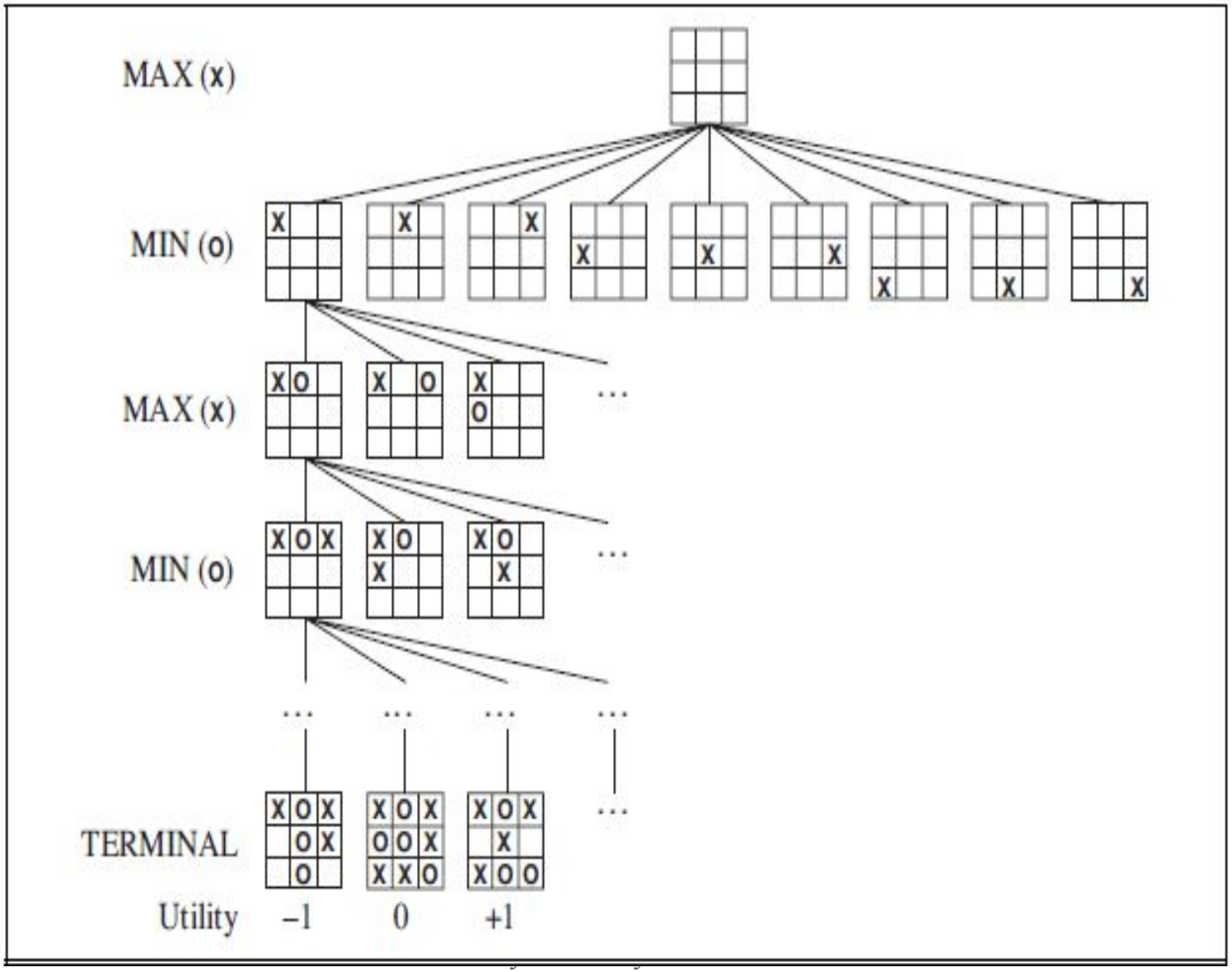
- A game can be formally defined as a kind of search problem with the following elements:
  1. S_0: The **initial state**, which specifies how the game is set up at the start.
  2. PLAYER(s): Defines which player has the move in a state.
  3. ACTIONS(s): Returns the set of legal moves in a state.
  4. RESULT(s, a): The **transition model**, which defines the result of a move.
  5. TERMINAL-TEST(s): A **terminal test**,
     - which is true when the game is over and
     - false otherwise.
     - States where the game has ended are called **terminal states**.
  6. UTILITY(s, p): A **utility function** defines the final numeric value for a game that ends in terminal state s for a player p.

Game Setup

- In chess,
  - The outcome is a win, loss, or draw, with values +1, 0, or ½
  - Chess is zero-sum (**all-or-nothing, win-or-lose)** because every game has payoff of either 0 + 1, 1 + 0 or 1/2+ ½

- The initial state, ACTIONS function, and RESULT function define the **game tree** for the game
  - a tree where
    - The nodes are game states and
    - The edges are moves.

Game Setup



Game Setup

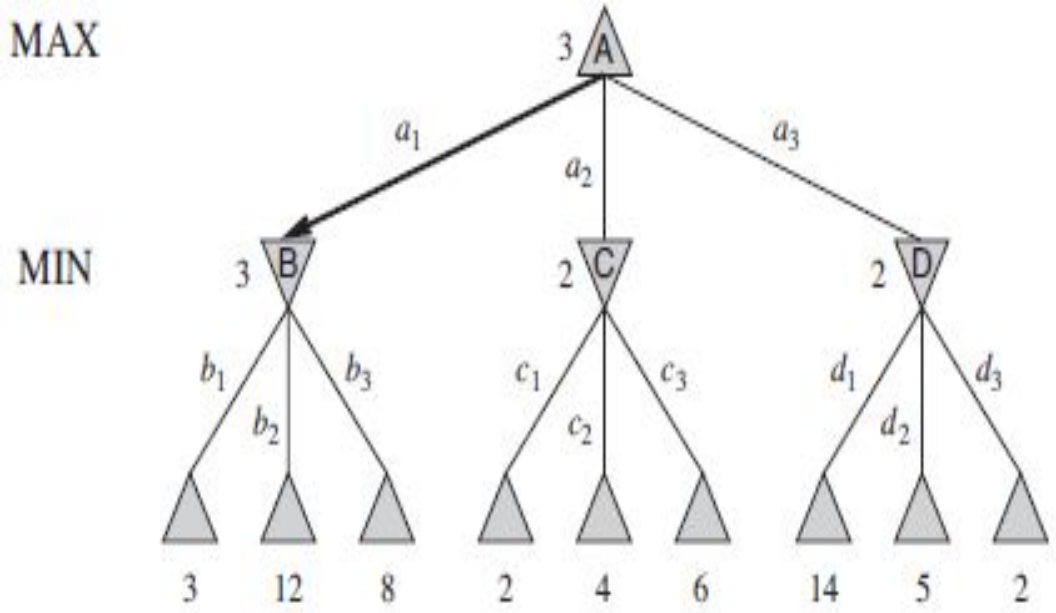
- Figure shows part of the game tree for tic-tac-toe
- From the initial state, MAX has nine possible moves.
- Play alternates between MAX's placing an X and MIN's placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled.
- The number on each leaf node indicates
  - The utility value of the terminal state from the point of view of MAX;
  - high values are assumed to be good for MAX and bad for MIN

Optimal Decisions In Games

- In a normal search problem
  - The optimal solution would be a sequence of actions leading to a goal state ie. a terminal state that is a win
- In adversarial search
  - MIN has something to say about it.
  - MAX therefore must find a contingent **strategy**,
  - which specifies MAX's move in the initial state,
  - then MAX's moves in the states resulting from every possible response by MIN,
  - then MAX's moves in the states resulting from every possible response by MIN to *those* moves, and so on.
  - This is exactly analogous to the AND–OR search algorithm with
    - MAX playing the role of OR and
    - MIN equivalent to AND.
  - Roughly speaking, an optimal strategy leads to outcomes at least as good as any other strategy when one is playing an infallible opponent.
  - We begin by showing how to find this optimal strategy.

Optimal Decisions In Games

- For game tic-tac-toe
  - possible moves for MAX at the root node are labeled a_1, a_2, and a_3.
  - The possible replies to a_1 for MIN are b_1, b_2, b_3, and so on.
  - This particular game ends after one move each by MAX and MIN.
  - The utilities of the terminal states in this game range from 2 to 14.



Optimal Decisions In Games

- Given a game tree, the optimal strategy can be determined from the **minimax value** of each node, which we write as MINIMAX(n).

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

- **How is utility calculated?**
- A utility function is usually defined by the designer. For example, we can assign scores based on the final board position.

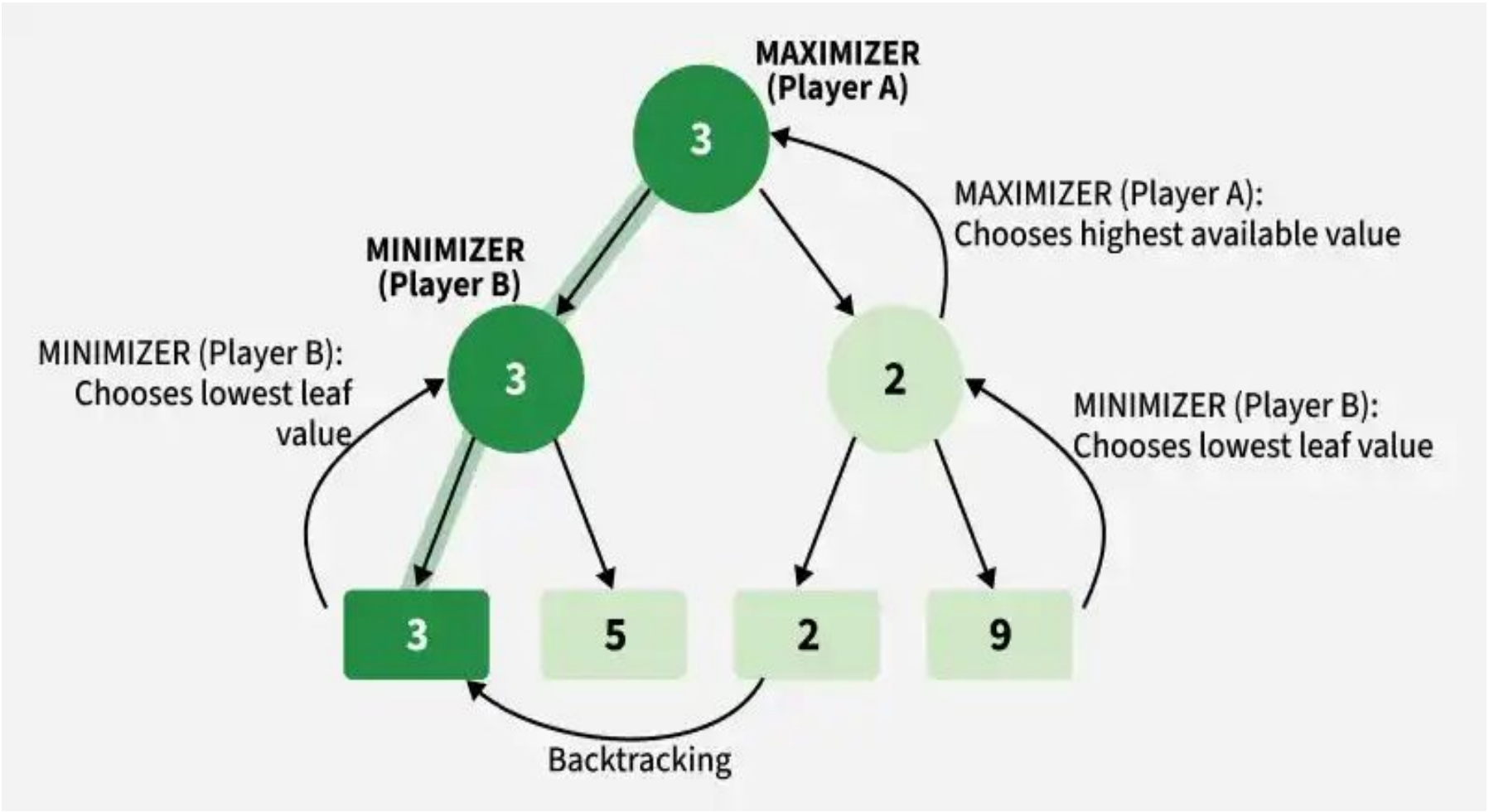
Optimal Decisions In Games

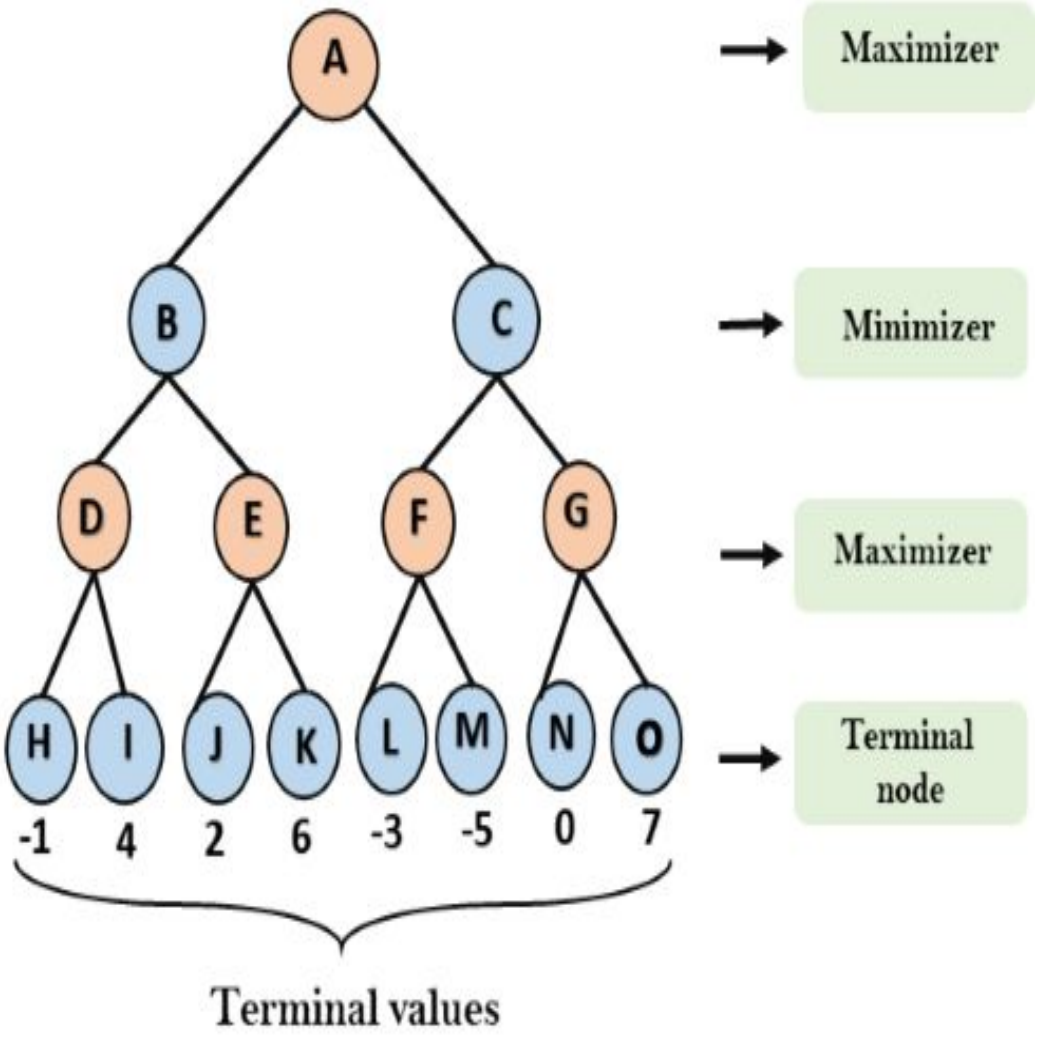
- The terminal nodes on the bottom level get their utility values from the game's UTILITY function.
- The first MIN node, labeled B, has three successor states with values 3, 12, and 8, so its minimax value is 3.
- Similarly, the other two MIN nodes have minimax value 2.
- The root node is a MAX node; its successor states have minimax values 3, 2, and 2; so it has a minimax value of 3.
- We can also identify the **minimax decision** at the root: action a1 is the optimal choice for MAX because it leads to the state with the highest minimax value.

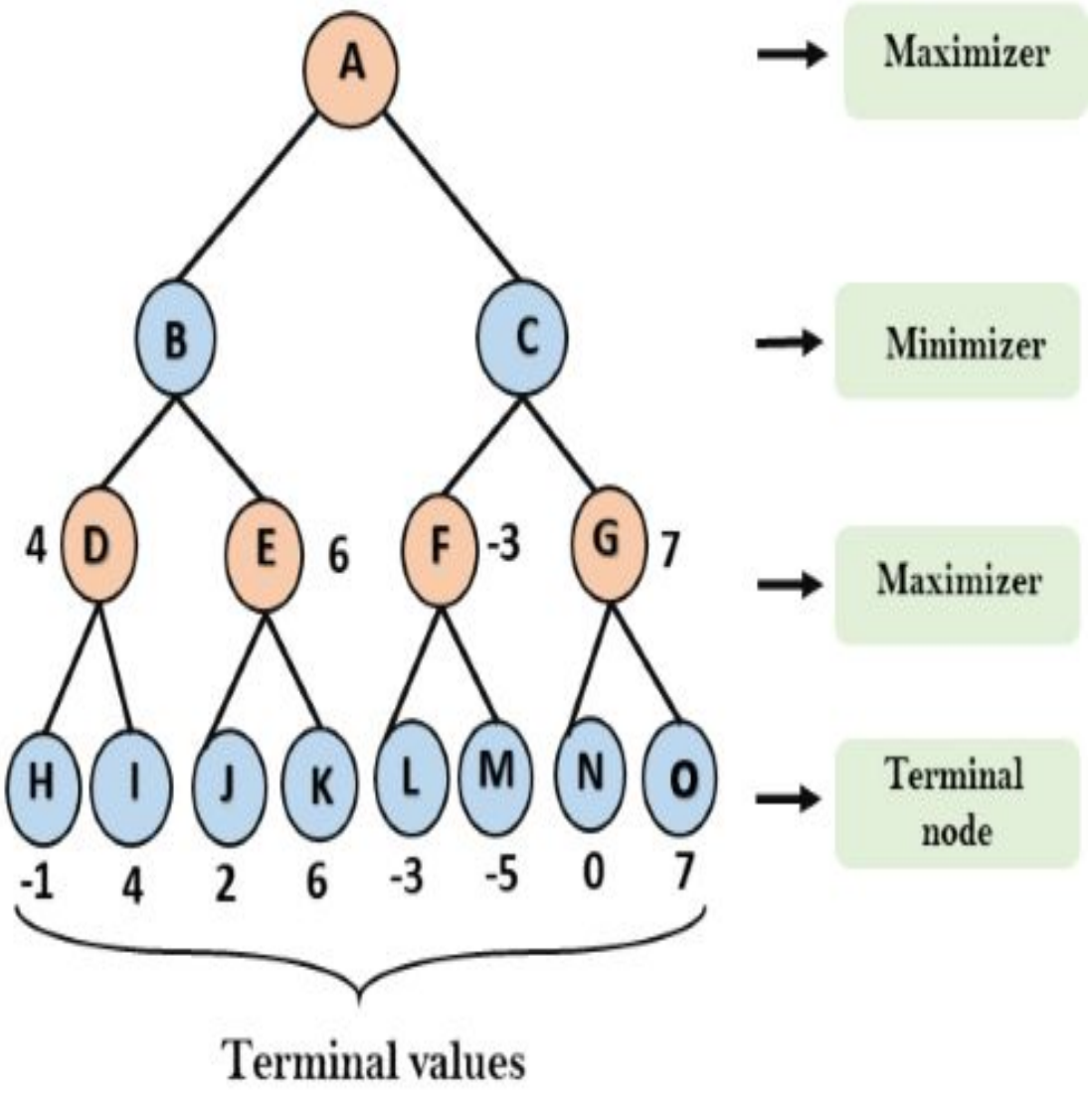
Minimax Algorithm

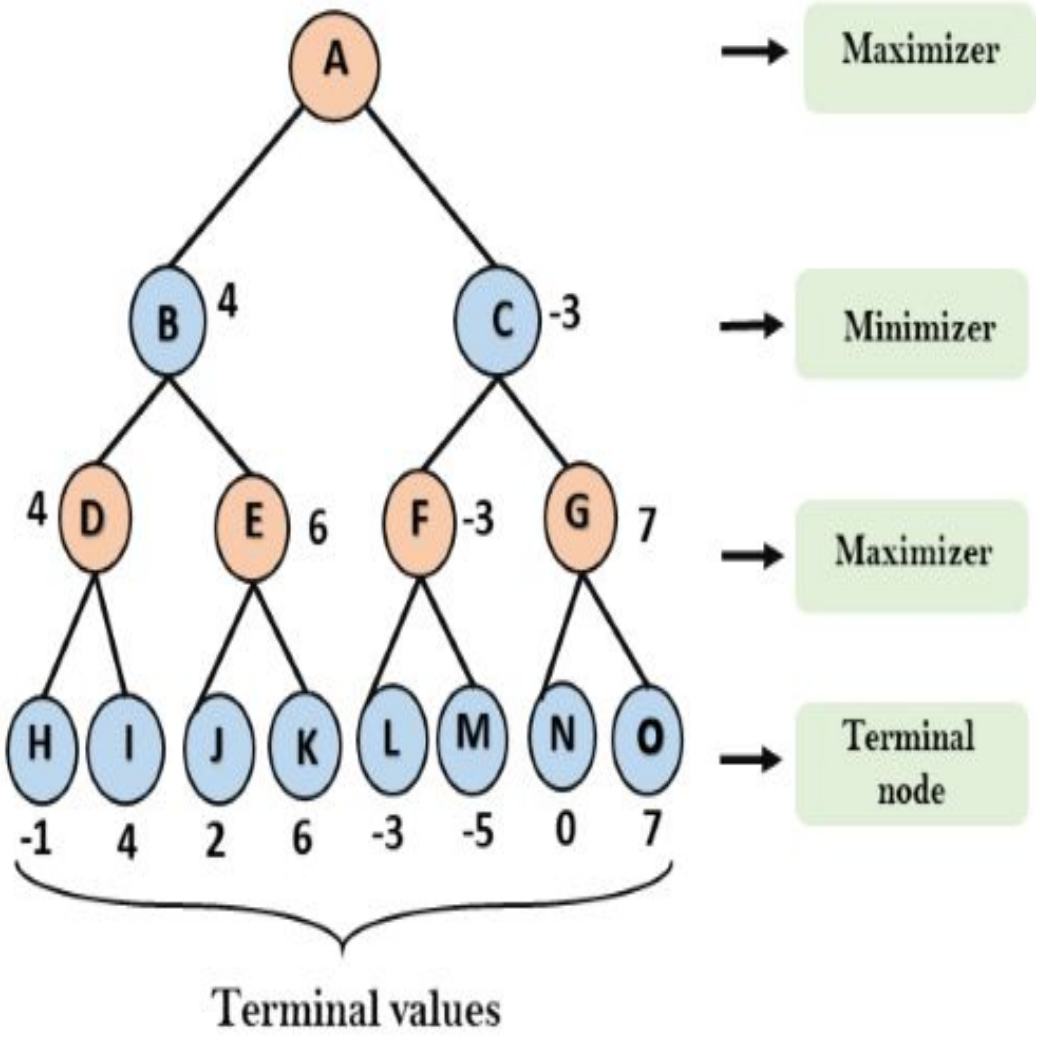
- The **minimax algorithm** computes the minimax decision from the current state.
- It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are **backed up** through the tree as the recursion unwinds.

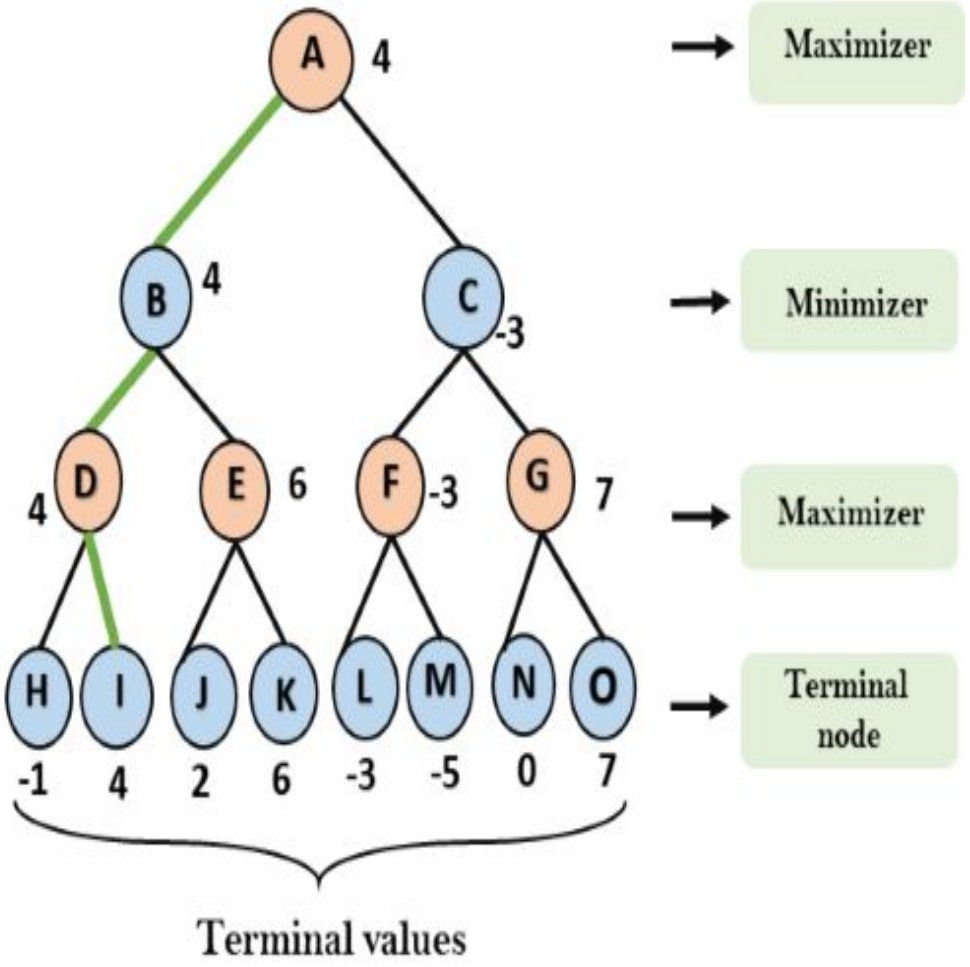
- Backtracking Algorithm
- Best Move Strategy
- Max will try to maximize its utility (Best Move)
- Min will try to minimize its utility (Worst Move)


MAXIMIZER
(Player A)
3
MAXIMIZER (Player A):
Chooses highest available value
MINIMIZER
(Player B)
MINIMIZER (Player B):
Chooses lowest leaf
value
3
2
MINIMIZER (Player B):
Chooses lowest leaf value
3
5
2
9
Backtracking


A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node


A
B
C
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node


A
B
4
C
-3
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values


A
4
B
4
C
-3
D
4
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Maximizer
Minimizer
Maximizer
Terminal node
Terminal values

Minimax Algorithm

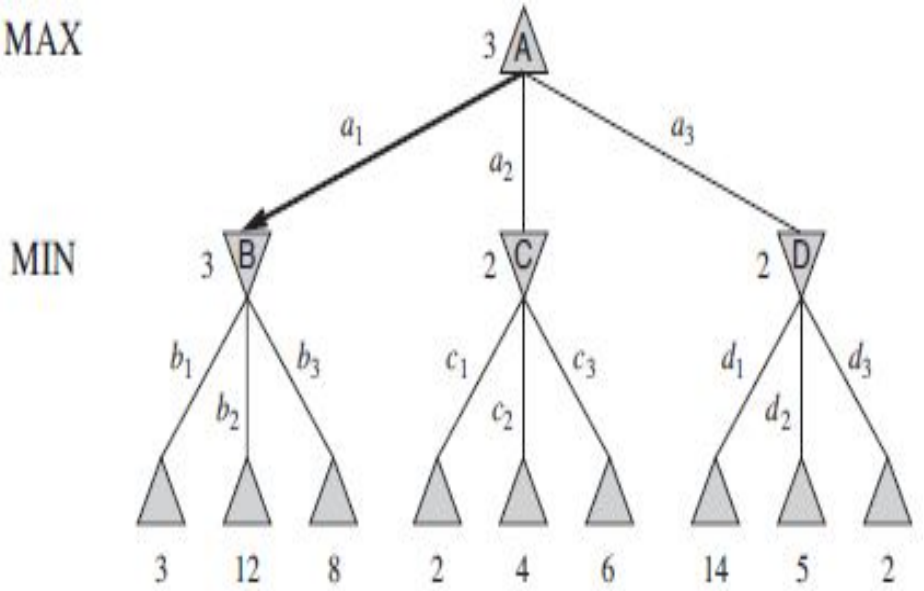
```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Minimax Algorithm

- For example, in Fig.,
  - The algorithm first recurses down to the three bottom left nodes and uses the UTILITY function on them to discover that their values are 3, 12, and 8, respectively.
  - Then it takes the minimum of these values, 3, and returns it as the backed up value of node B.
  - A similar process gives the backed-up values of 2 for C and 2 for D.
  - Finally, we take the



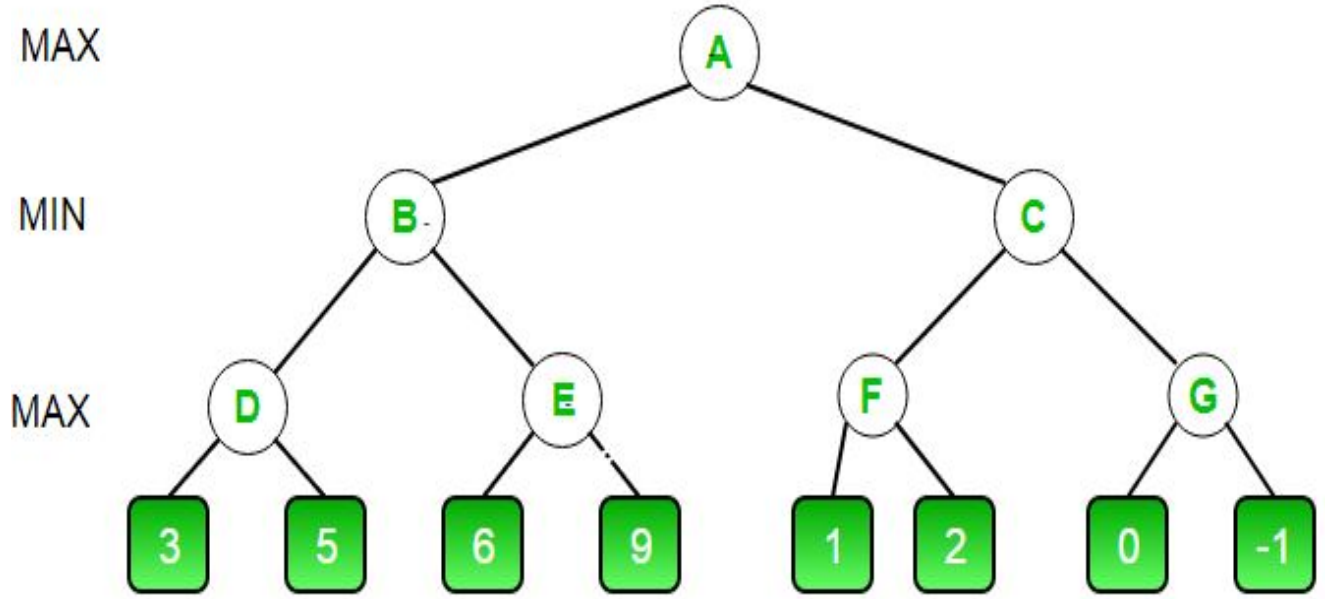
Minimax Algorithm

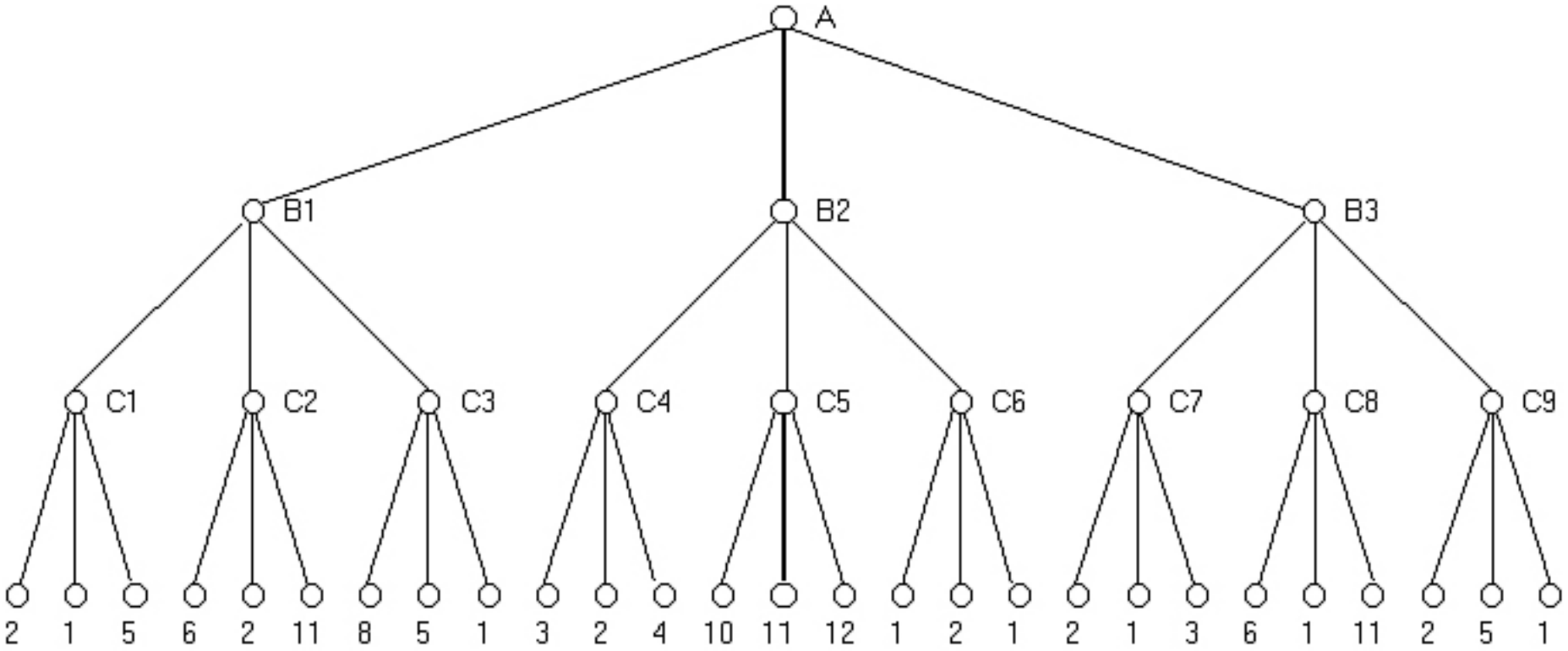
- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If the maximum depth of the tree is m and b legal moves at each point,
  - **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
  - **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
  - **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **$O(b^m)$**, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
  - **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS which is **O(bm)**.

Limitation of the minimax Algorithm:

- **The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of games has a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from alpha-beta pruning which we have discussed in the next topic.**

Solve




A
B1
B2
B3
C1
C2
C3
C4
C5
C6
C7
C8
C9
2 1 5 6 2 11 8 5 1 3 2 4 10 11 12 1 2 1 2 1 3 6 1 11 2 5 1

Alpha–Beta Pruning

1. The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
2. we can't eliminate the exponent, but it turns out we can effectively cut it in half.
3. The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.
4. Idea of **pruning** to eliminate large parts of the tree from consideration.
5. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
6. When applied to a standard minimax tree,
    1. It returns the same move as minimax would,
    2. But prunes away branches that
    3. Cannot possibly influence the final decision.

- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.

- The two-parameter can be defined as:
  - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is -∞.

  - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞.

- The main condition which required for alpha-beta pruning is:α>=β

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

Alpha–Beta Pruning

function minimax(node, depth, alpha, beta, maximizingP
layer) is

if depth ==0 or node is a terminal node then

return static evaluation of node

```
// for Maximizer Player

if MaximizingPlayer then
    maxEva= -infinity
    for each child of node do
    eva= minimax(child, depth-1,
      alpha, beta, False)
   maxEva= max(maxEva, eva)

   alpha= max(alpha, maxEva)

    if beta<=alpha
  break
  return maxEva
```

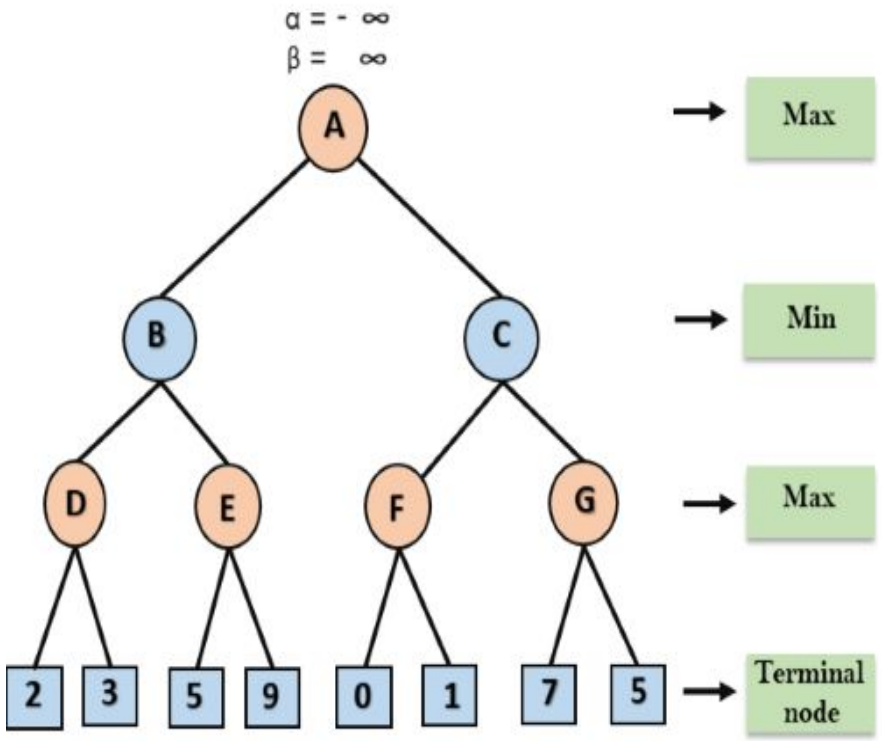
```
// for Minimizer    player

else
    minEva= +infinity
    for each child of node do

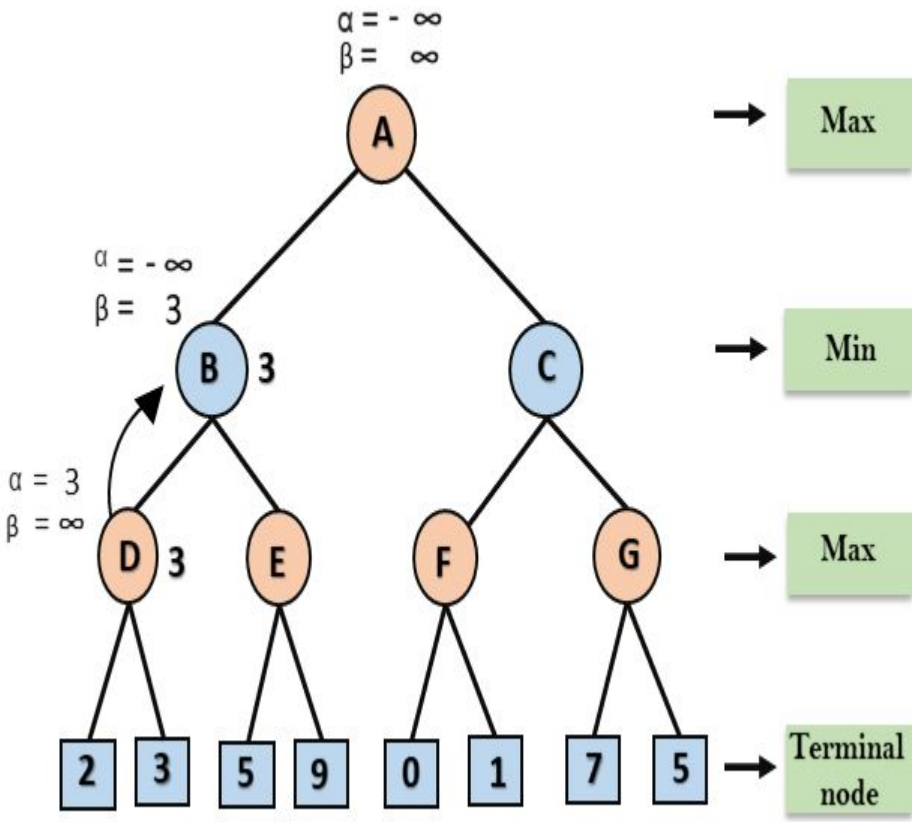
    eva= minimax(child, dept
      h-1, alpha, beta, true)
    minEva= min(minEva, eva
      )
    beta= min(beta, eva)
     if beta<=alpha
   break
  return minEva
```

Working of Alpha-Beta Pruning:

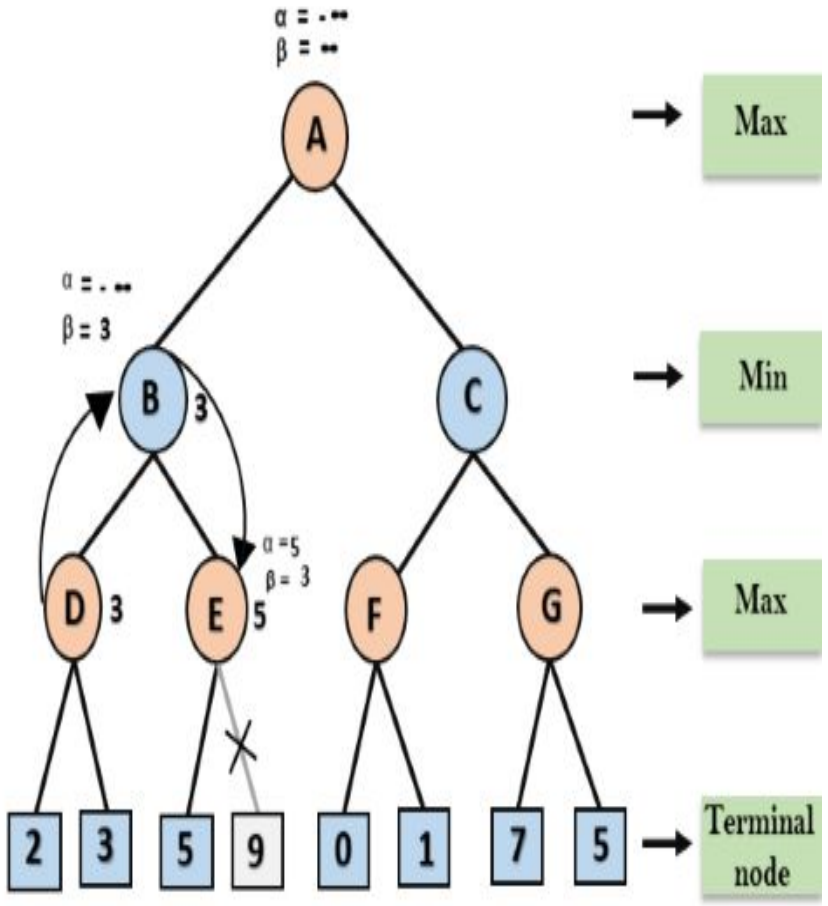
- **Step 1:** At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child



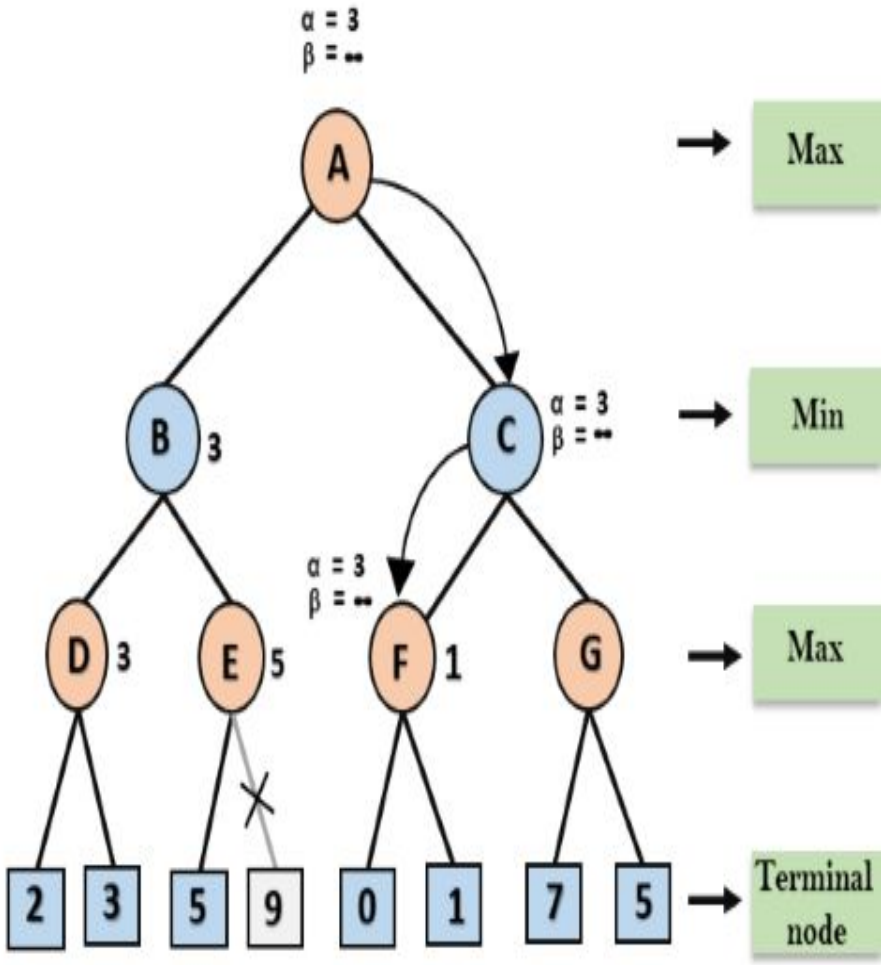
- **Step 2:** At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.
- **Step 3:** Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3



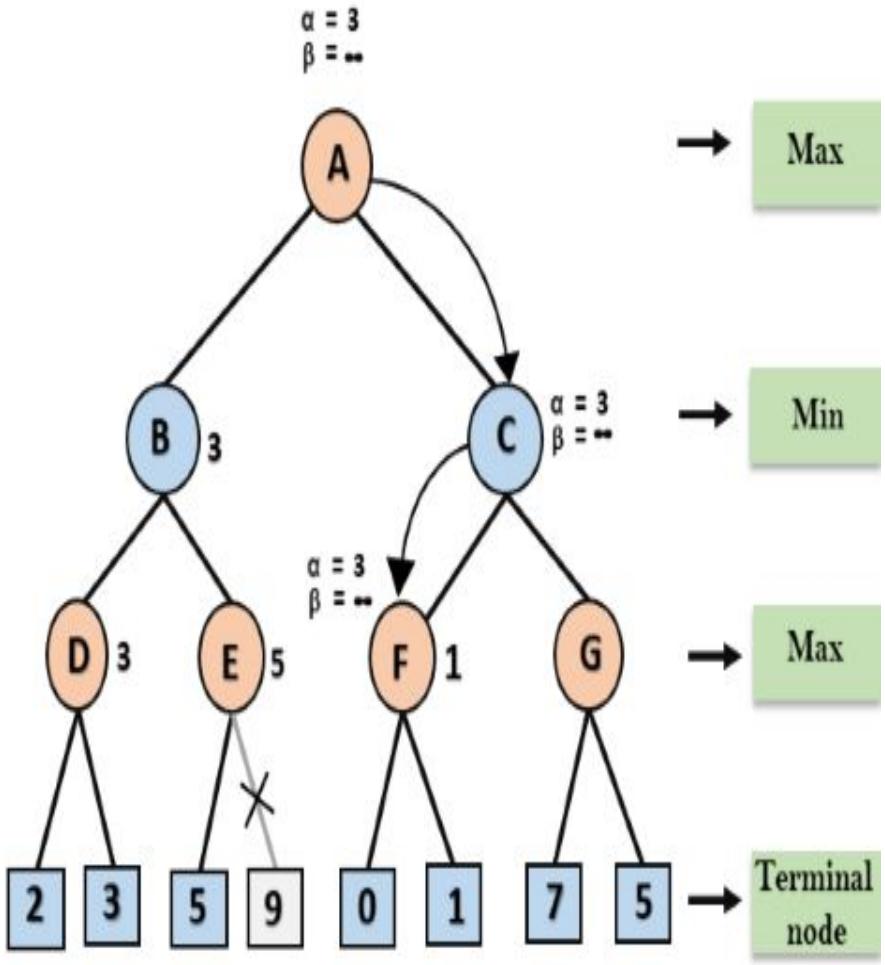
- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.
- **Step 4:** At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where **α>=β**, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



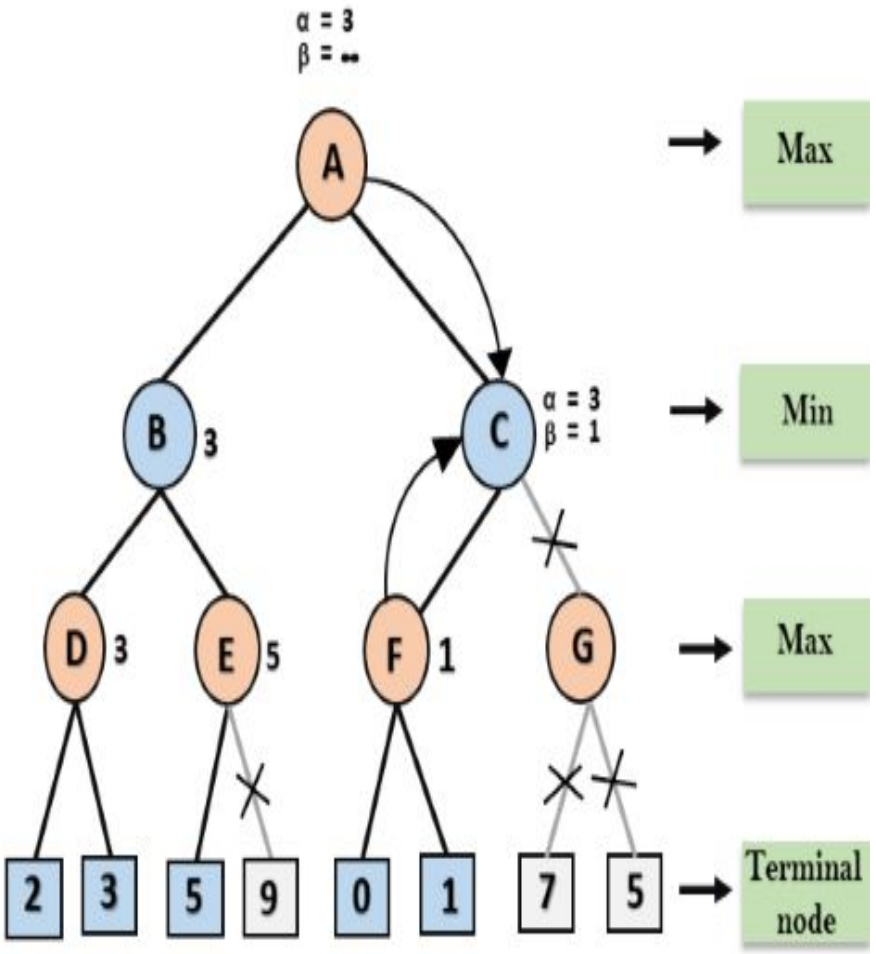
- **Step 5:** At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.
- At node C, α=3 and β= +∞, and the same values will be passed on to node F.



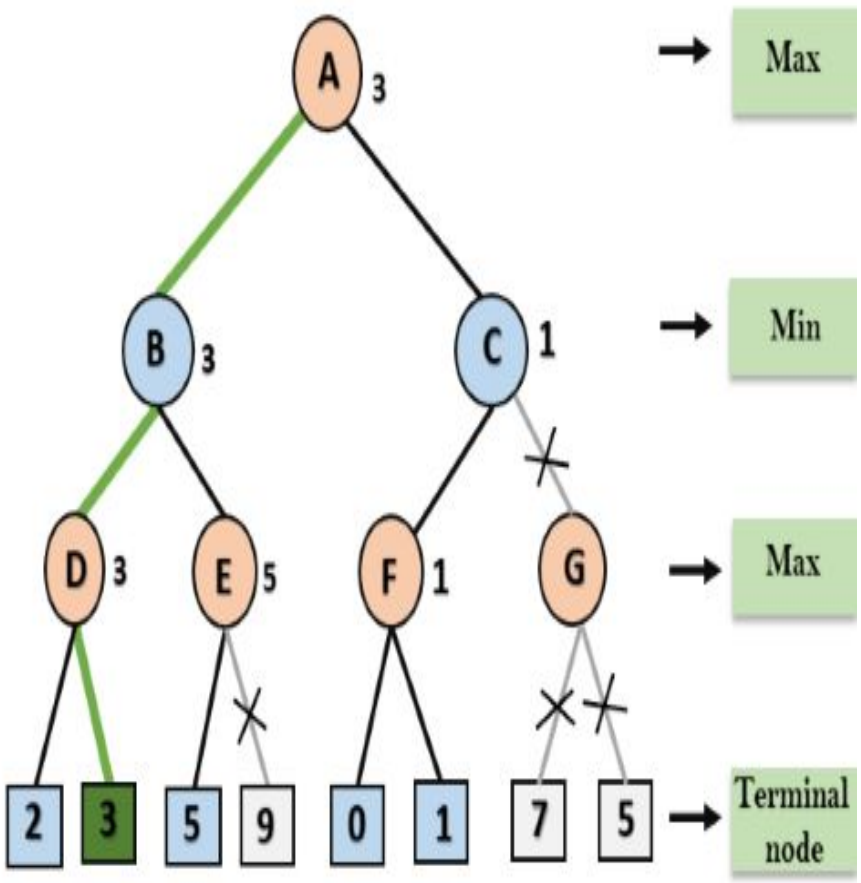
- **Step 6:** At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition **α>=β,** so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



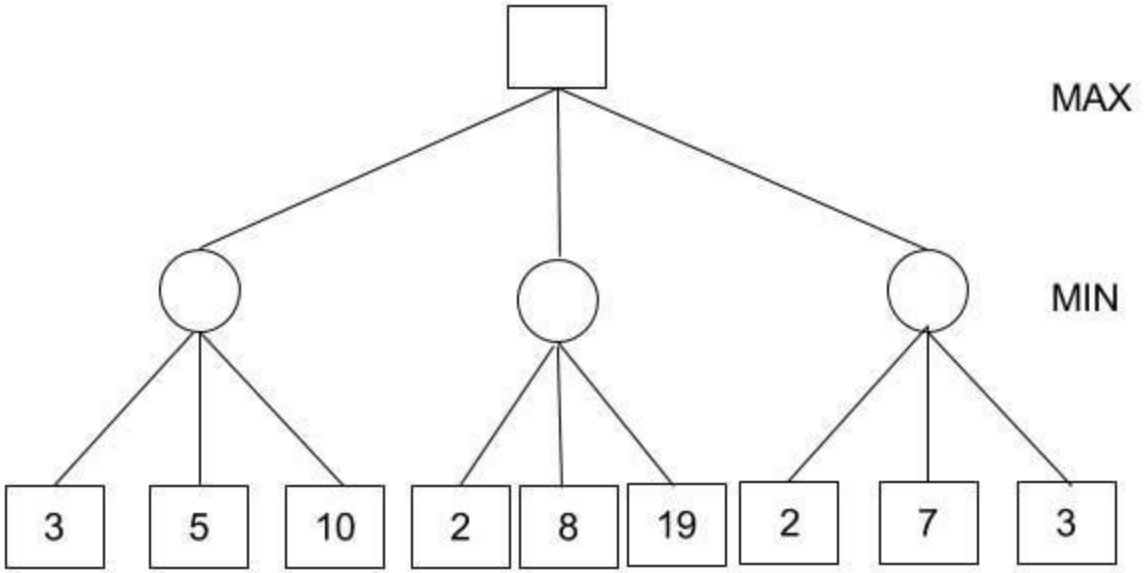
Properties of α-β

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$
- A simple example of the value of reasoning about which computations are relevant (a form of metareasoning)

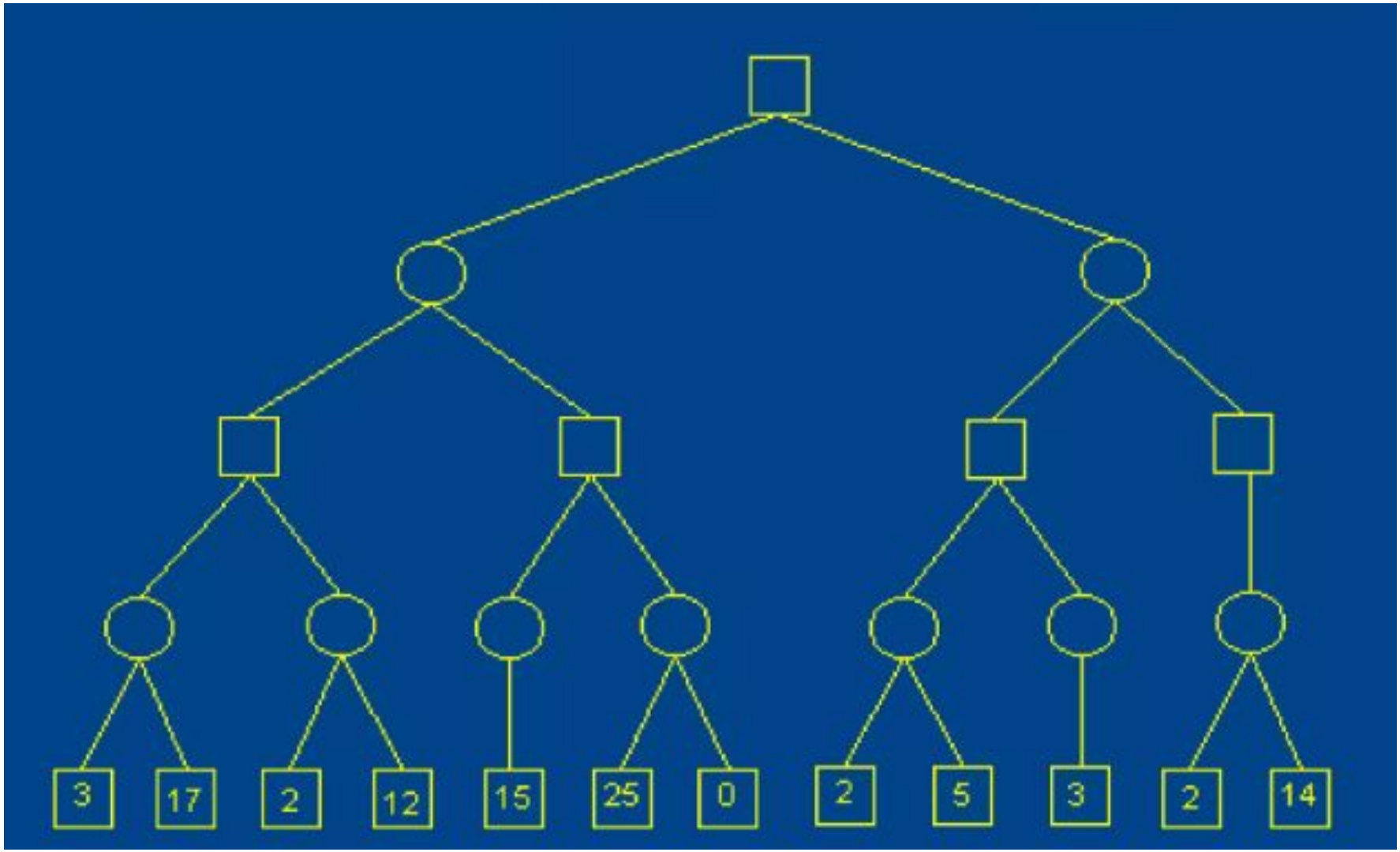
Move Ordering in Alpha-Beta pruning:

- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.

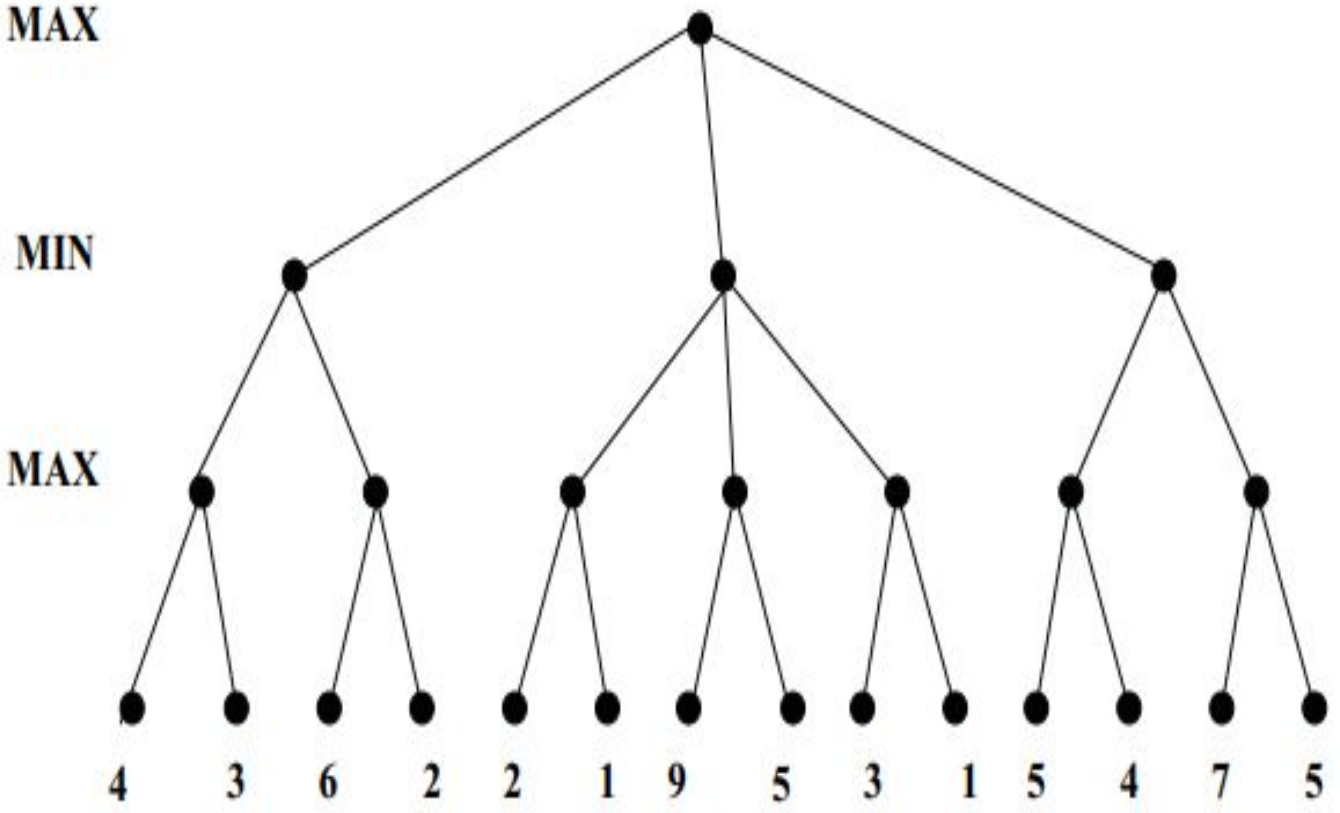
Solve



Solve



Alpha beta pruning. Example



Monte Carlo Tree Search (MCTS)

- Now we understand the fact that we need something **smarter** than uninformed search to navigate through a gigantic state space like the game of Go.

- Monte Carlo Tree Search (MCTS) is a heuristic search set of rules that has won big attention and reputation within the discipline of synthetic intelligence, specially in the area of choice-making and game playing.

Monte Carlo Tree Search (MCTS)

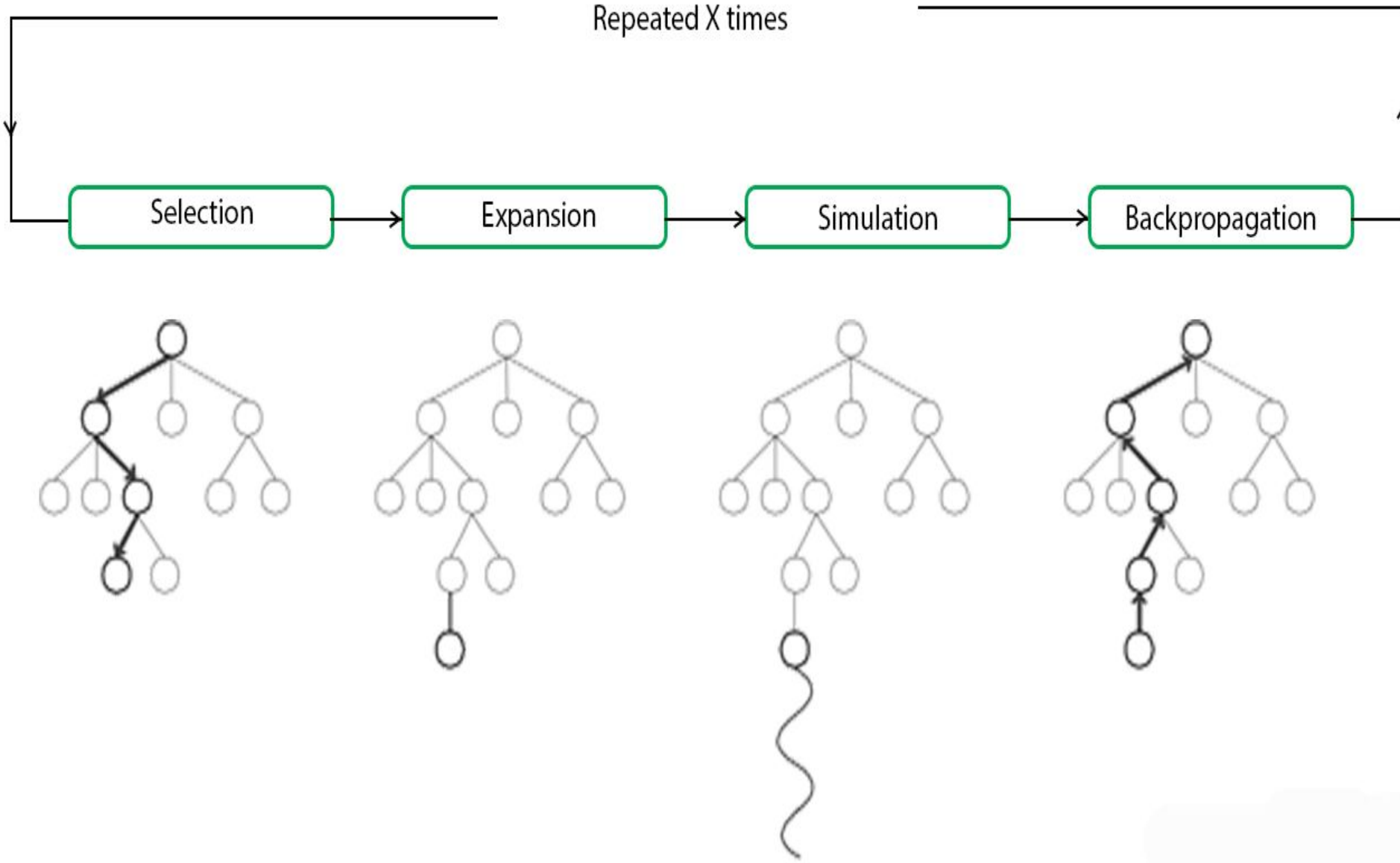
- Ability to effectively handle complex and strategic video games with massive search areas, in which traditional algorithms may additionally struggle due to the full-size number of feasible actions or actions.
- MCTS has been efficiently implemented in numerous domains, including board games (e.G., Go, chess, and shogi), card video games (e.G., poker), and video games.
- One of the exquisite blessings of MCTS is its ability to handle video games with unknown or imperfect data, as it relies on statistical sampling as opposed to whole know-how of the game state.

Monte Carlo Tree Search (MCTS)

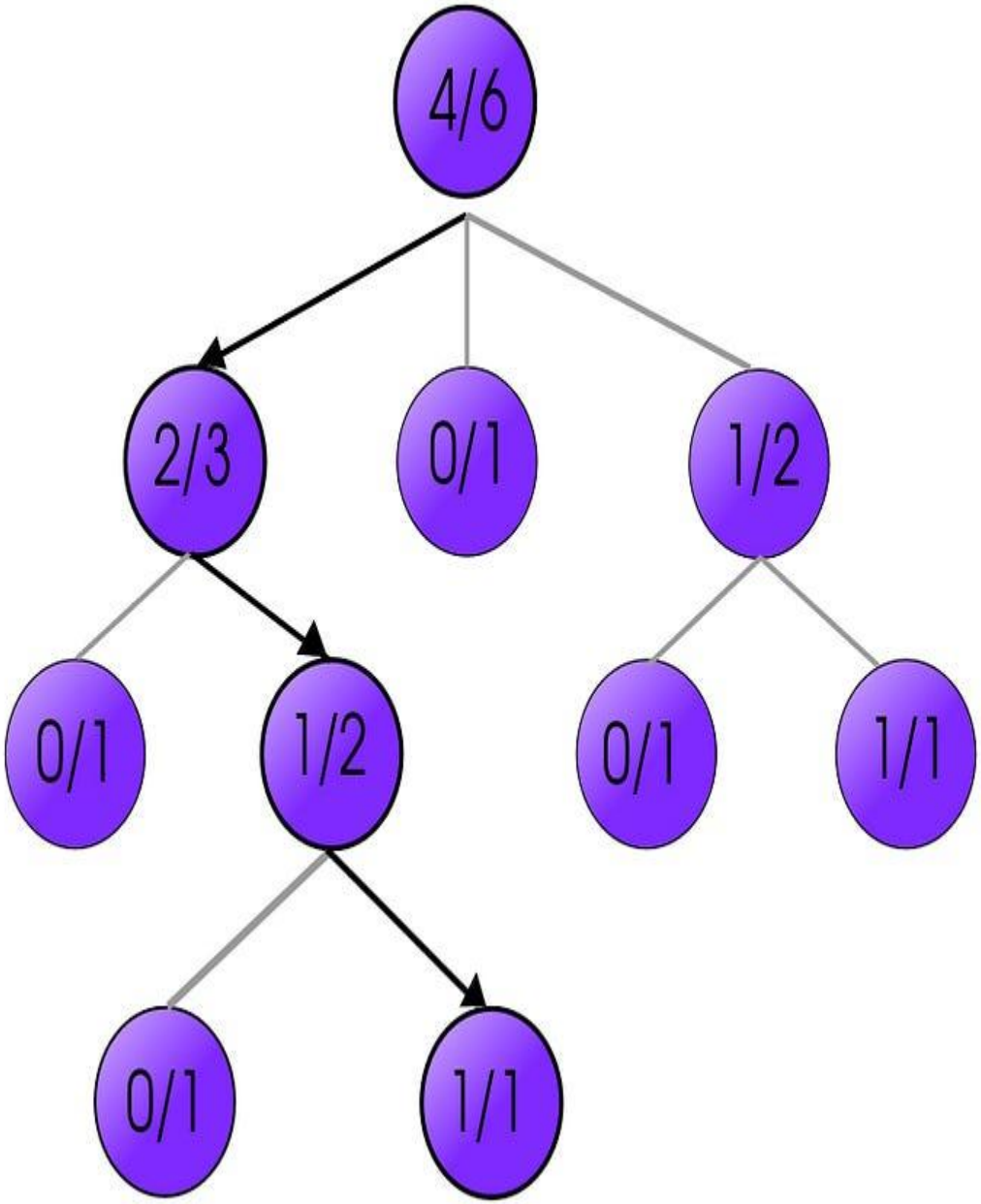
- In tree search, there's always the possibility that the current best action is actually not the most optimal action. In such cases, MCTS algorithm becomes useful as it continues to evaluate other alternatives periodically during the learning phase by executing them, instead of the current perceived optimal strategy. This is known as the " ***exploration-exploitation trade-off*** ".
- It **exploits** the actions and strategies that is found to be the best till now but also must continue to **explore** the local space of alternative decisions and find out if they could replace the current best.

Why use Monte Carlo Tree Search (MCTS) ?

- Handling Complex and Strategic Games
- Unknown or Imperfect Information
- Learning from Simulations
- Optimizing Exploration and Exploitation
- Scalability and Parallelization
- Applicability Beyond Games
- Domain Independence


Repeated X times
Selection
Expansion
Simulation
Backpropagation

- **Selection:** In this process, the MCTS algorithm traverses the current tree from the root node using a specific strategy. The strategy uses an evaluation function to optimally select nodes with the highest estimated value. MCTS uses the Upper Confidence Bound (UCB) formula applied to trees as the strategy in the selection process to traverse the tree.


4/6
2/3
0/1
1/2
0/1
1/2
0/1
1/1
0/1
1/1

- **Selecting**👆| This process is used to select a node on the tree that has the highest possibility of winning. For Example — Consider the moves with winning possibility 2/3, 0/1 & 1/2 after the first move 4/6, the node **2/3** has the highest possibility of winning.
- The node selected is searched from the current state of the tree and selected node is located at the end of the branch. Since the selected node has the highest possibility of winning — that path is also most likely to reach the solution faster than other path in the tree.

$$S_i = x_i + C\sqrt{\frac{\ln(t)}{n_i}}$$

- where;
  S_i = value of a node i
  x_i = empirical mean of a node i
  C = a constant
  t = total number of simulations
  When traversing a tree during the selection process, the child node that returns the greatest value from the above equation will be one that will get selected. During traversal, once a child node is found which is also a leaf node, the MCTS jumps into the expansion step.

for node ⅔

Xi=2/3

C=any constant ex-100

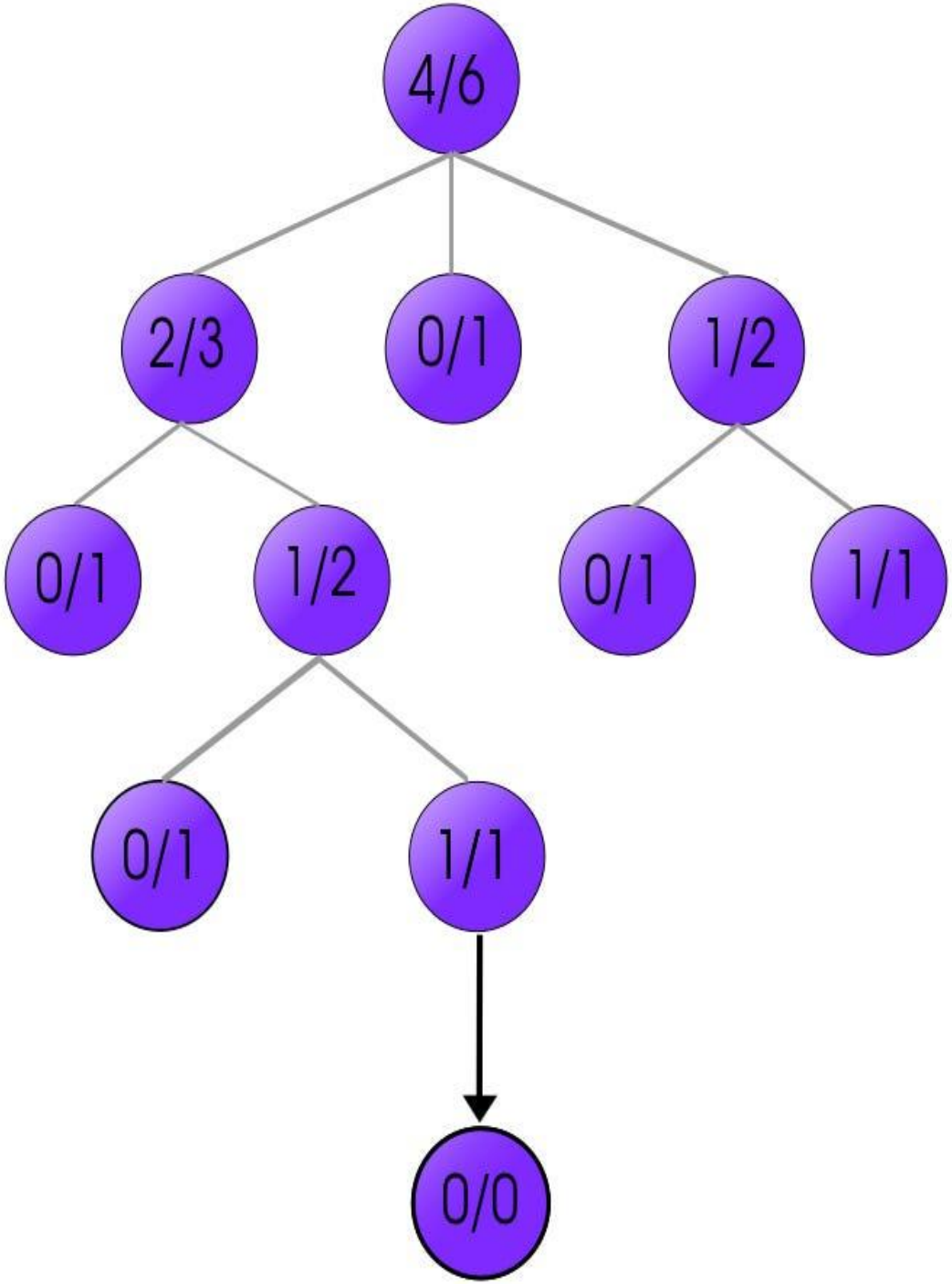
t=6

ni= 3

Si= ⅔ +100 $\sqrt{\ln(6)/3}$

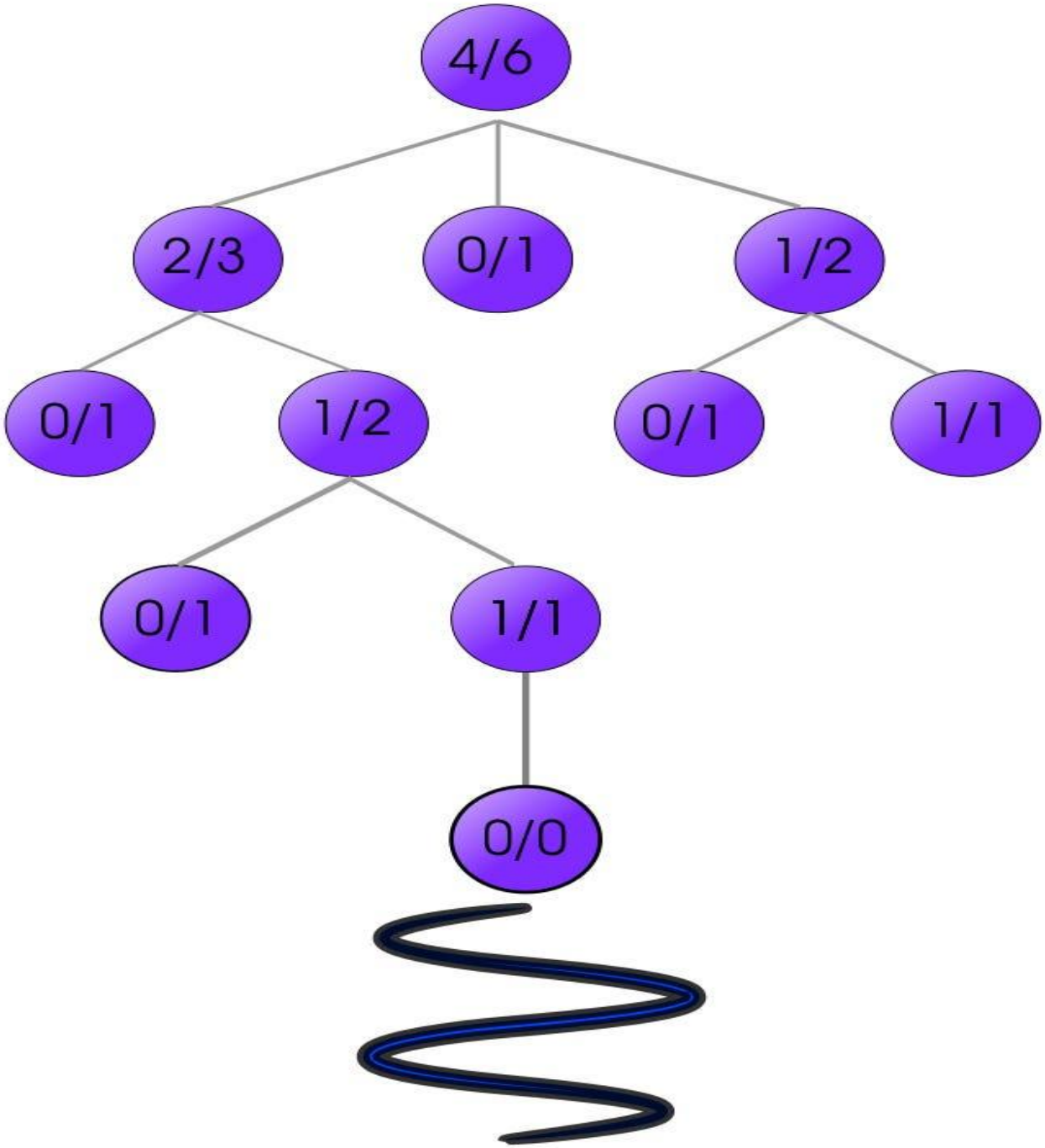
Si=77.94

- **Expansion:** In this process, a new child node is added to the tree to that node which was optimally reached during the selection process.


4/6
2/3
0/1
1/2
0/1
1/2
0/1
1/1
0/1
1/1
0/0

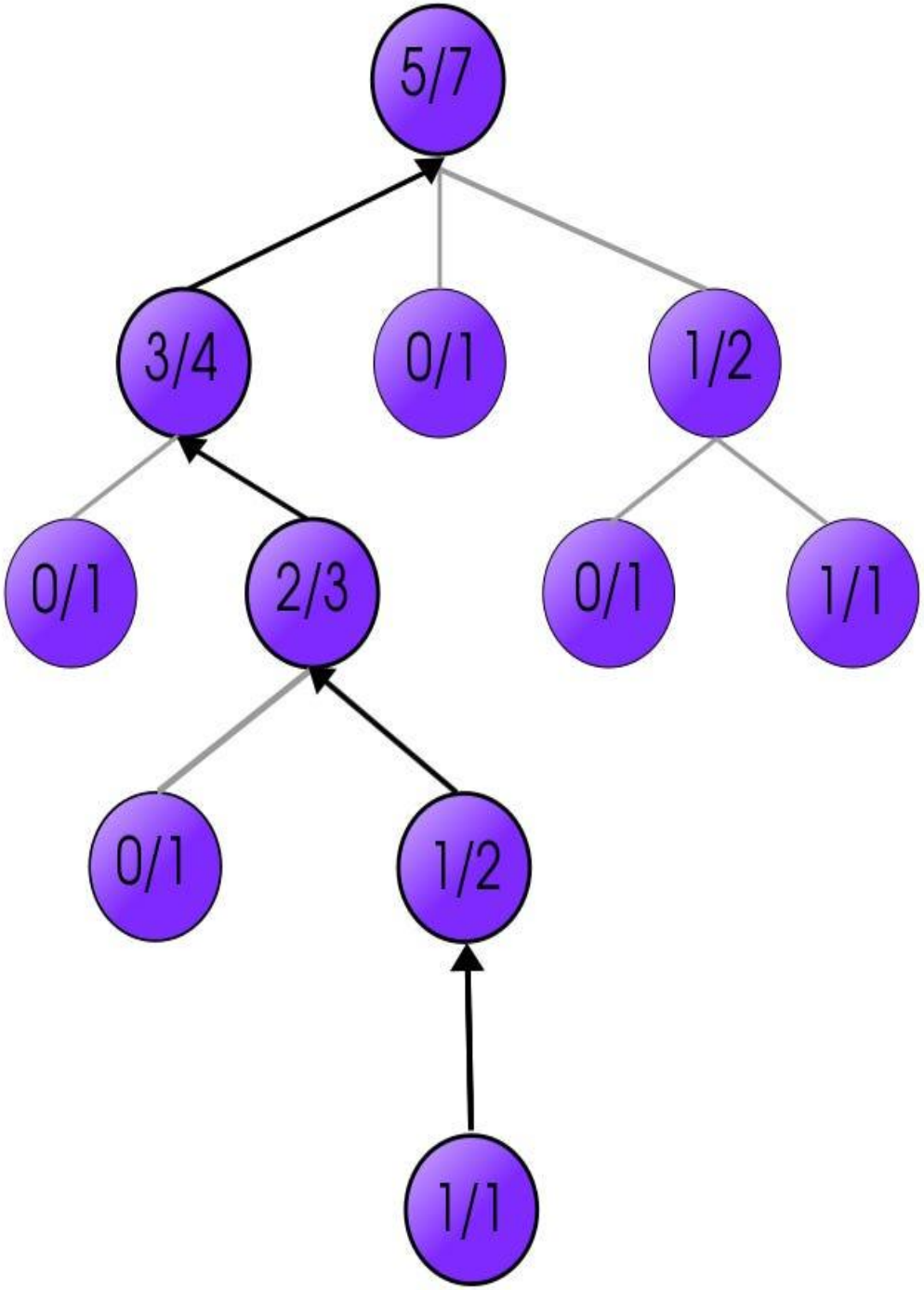
- **Expanding —** After selecting the right node. Expanding is used to increase the options further in the game by expanding the selected node and creating many children nodes. We are using only one children node in this case. These children nodes are the future moves that can be played in the game.
- The nodes that are not expanded further for the time being are known are leaves.

- **Simulation:** In this process, a simulation is performed by choosing moves or strategies until a result or predefined state is achieved.
- The simulation is done for every children node is followed by their individual rewards.


4/6
2/3
0/1
1/2
0/1
1/2
0/1
1/1
0/1
1/1
0/0

- **Simulating|Exploring** 🚀 Since nobody knows which node is the best children/ leaf from the group. The move which will perform best and lead to the correct answer down the tree.
- ***How do we find the best children which will lead us to the correct solution?***
- We use Reinforcement Learning to make random decisions in the game further down from every children node. Then, reward is given to every children node — by calculating how close the output of their random decision was from the final output that we need to win the game.
- **For example:** In the game of Tic-Tac-Toe. Does the random decision to make cross(X) next to previous cross(X) in the game results in three consecutive crosses(X-X-X) that are needed to win the game?

- **Backpropagation:** After determining the value of the newly added node, the remaining tree must be updated. So, the backpropagation process is performed, where it backpropagates from the new node to the root node. During the process, the number of simulation stored in each node is incremented. Also, if the new node's simulation results in a win, then the number of wins is also incremented.


5/7
3/4
0/1
1/2
0/1
2/3
0/1
1/1
0/1
1/2
1/1

- Let’s say the simulation of the node gives optimistic results for its future and gets a positive score 1/1.
- **Updating|Back-propagation —** Due to the new nodes and their *positive or negative* scores in the environment. The total scores of their parent nodes must be updated by going back up the tree one-by-one. The new updated scores changes the state of the tree and may also change new future node of the selection process.

MCTS Tic-Tac-Toe Example

Monte Carlo Tree Search (MCTS) is a way for an AI to choose a move by repeatedly simulating possible games.

Suppose the board is:

```
X | O | X
---------
  | O |
---------
  | X |
```

It is O's turn.

1. Selection

MCTS starts at the current position and selects a promising move according to a formula such as:

$$UCT = \frac{W_i}{N_i} + C\sqrt{\frac{\ln N}{N_i}}$$

where:

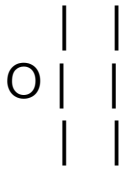
Wi = wins from child node i

Ni = number of times child i was visited

N = visits to the parent

C = exploration constant

Possible moves for O might be:







and so on.

2. Expansion

Suppose MCTS selects the center-left square:

```
X | O | X
---------
O | O |
---------
  | X |
```

This new position becomes a new node in the search tree.

3. Simulation (Rollout)

From that position, MCTS plays the rest of the game using random or semi-random moves.

For example:

```
X | O | X
---------
O | O | X
---------
X | X | O

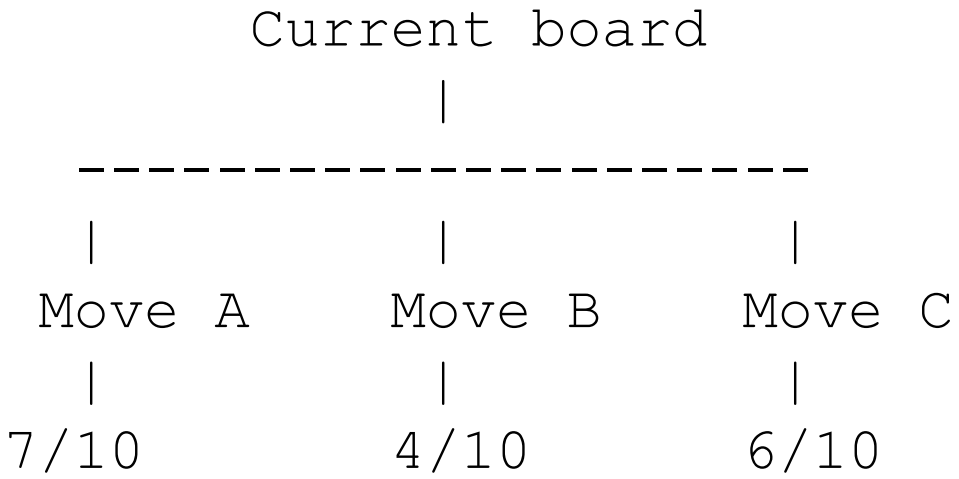
O wins.

So this simulation gives a result such as:
Reward=1
for O.
```

4. Backpropagation

The result is propagated back up the tree.

For example:



Move	Simulations	O Wins	Win rate
A	100	70	70%
B	100	40	40%
C	100	60	60%

Therefore MCTS chooses Move A.

The four MCTS steps

Selection→Expansion→Simulation→Backpropagation

The algorithm repeats these steps hundreds or thousands of times and then chooses the move with the best statistics.

For Tic-Tac-Toe, MCTS is small enough that you can actually draw the complete search tree and calculate the UCT values by hand.

MCTS Tic-tac-toe (Monte Carlo Tree Search) Visualization

https://vgarciasc.github.io/mcts-viz/

Key Points of MCTS:

- **Exploration vs Exploitation**: MCTS ensures that the algorithm explores various moves, but also biases toward moves that appear to have high chances of success, given prior simulations.
- **Statistical Process**: Over time, the tree grows and refines itself, using the results of random simulations to approximate the best possible move.
- **Iterative**: The process of selection, expansion, simulation, and backpropagation is repeated many times. With each iteration, the algorithm gets closer to identifying the most optimal move.

MCTS is incredibly effective for large and complex search spaces (such as Go or other strategic games) where traditional methods, like minimax, are computationally prohibitive. The more simulations you run, the better your understanding of the

Advantages of Monte Carlo Tree Search:

- MCTS is a simple algorithm to implement.
- Monte Carlo Tree Search is a heuristic algorithm. MCTS can operate effectively without any knowledge in the particular domain, apart from the rules and end conditions, and can find its own moves and learn from them by playing random playouts.
- The MCTS can be saved in any intermediate state and that state can be used in future use cases whenever required.
- MCTS supports asymmetric expansion of the search tree based on the circumstances in which it is operating.

Disadvantages of Monte Carlo Tree Search:

- As the tree growth becomes rapid after a few iterations, it requires a huge amount of memory.
- There is a bit of a reliability issue with Monte Carlo Tree Search. In certain scenarios, there might be a single branch or path, that might lead to loss against the opposition when implemented for those turn-based games. This is mainly due to the vast amount of combinations and each of the nodes might not be visited enough number of times to understand its result or outcome in the long run.
- MCTS algorithm needs a huge number of iterations to be able to effectively decide the most efficient path. So, there is a bit of a speed issue there.

Stochastic Games

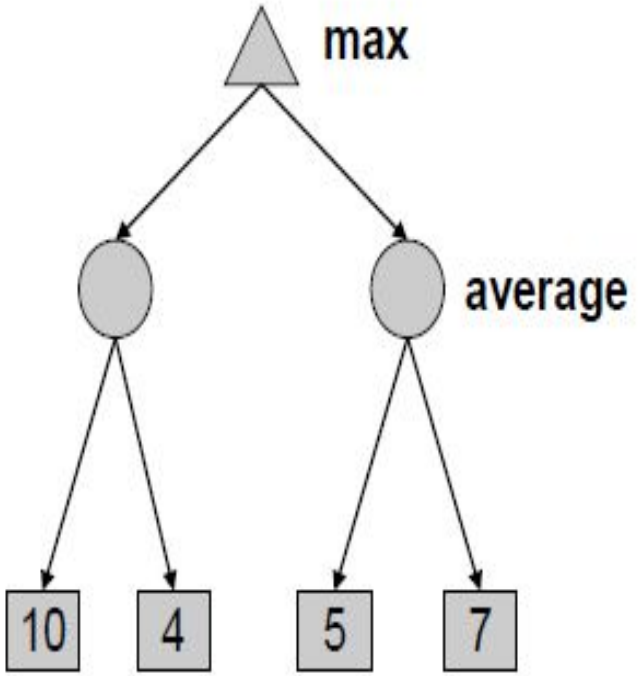
By Mrs. Priyadarshani Doke.

Stochastic games in artificial intelligence represent a sophisticated intersection of game theory, probability, and decision-making, capturing the inherent uncertainty found in real-world interactions. Unlike deterministic games, where outcomes are predictable based on the actions taken, stochastic games introduce randomness, allowing AI to navigate unpredictable environments. Whether through dice rolls or random events, stochastic games enable AI systems to handle uncertainty while making decisions.

Stochastic Single-Player

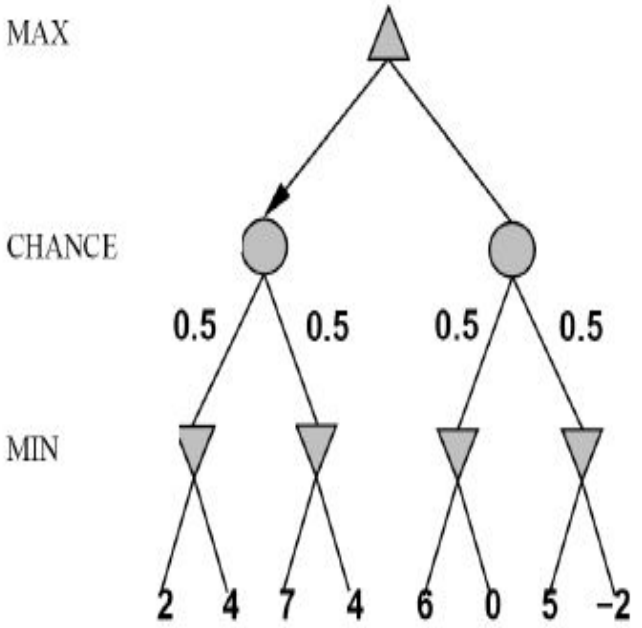
- What if we don't know what the result of an action will be? E.g.,
  - In solitaire, shuffle is unknown
  - In minesweeper, mine locations
  - In pacman, ghosts!

- Can do **expectimax search**
  - Chance nodes, like actions except the environment controls the action chosen
  - Calculate utility for each node
  - Max nodes as in search
  - Chance nodes take average (expectation) of value of children



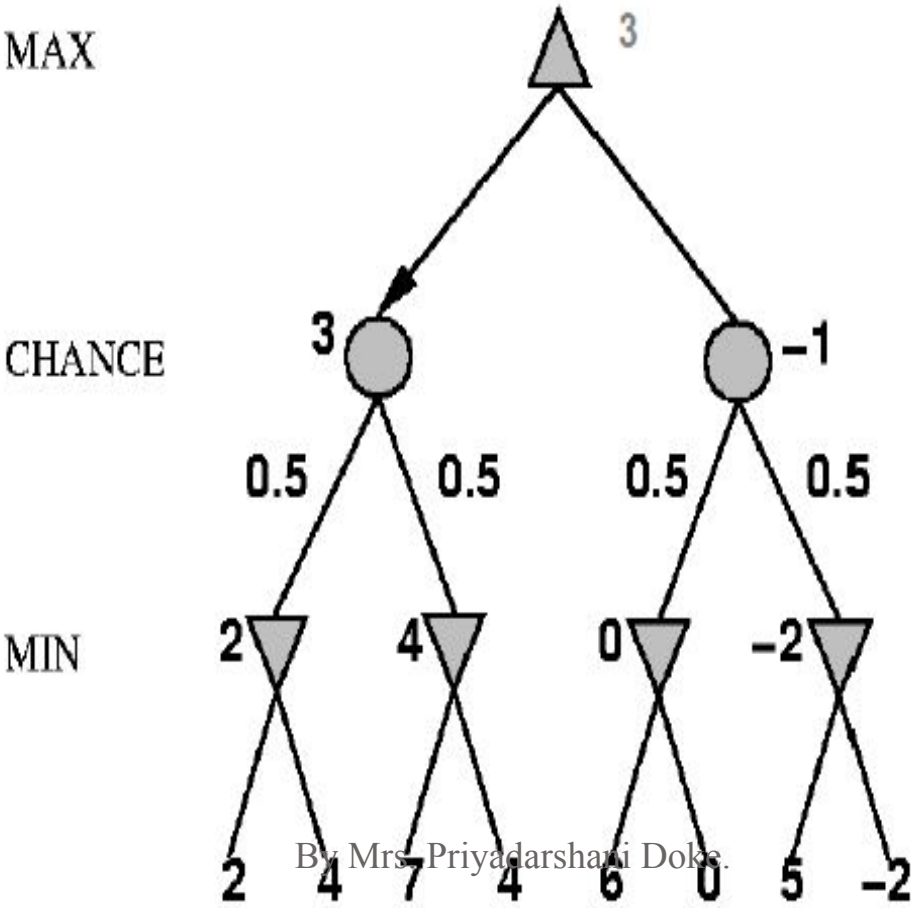
Stochastic Two-Player

- E.g. backgammon
- Expectiminimax (!)
  - Environment is an extra player that moves after each agent
  - Chance nodes take expectations, otherwise like minimax



if *state* is a MAX node **then**
    return the highest EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)
if *state* is a MIN node **then**
    return the lowest EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)
if *state* is a chance node **then**
    return average of EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)

Minimax for nondeterministic games



Expectiminimax Algorithm

The **expectiminimax algorithm** is a generalization of the minimax algorithm used for stochastic games. It takes into account not only the decisions of two competing players (MAX and MIN), but also the influence of random events (CHANCE).

The algorithm proceeds as follows:

EXPECTIMINIMAX (s)=

UTILITY (s)

max a EXPECTIMINIMAX (RESULT(s,a))

min a EXPECTIMINIMAX(RESULT(s,a))

$\sum r\ P(r)$EXPECTIMINIMAX(RESULT(s,r))

if TERMINAL-TEST (s)

if PLAYER (s)=MAX

if PLAYER(s)=MIN

if PLAYER(s)=CHANCE

Where:

- **UTILITY(s):** The utility or value of state s (i.e., the outcome value of a terminal state).
- **MAX:** The player trying to maximize their payoff (e.g., White).
- **MIN:** The player trying to minimize the opponent's payoff (e.g., Black).
- **CHANCE:** The chance node, where the outcome is determined by a random event (like rolling dice).
- **RESULT(s, r):** The state s after taking into account the result of a random event r (e.g., dice roll).

Limitations of Game Search Algorithms

- **Exponential Growth**: As the game progresses, the number of possible moves and game states can grow exponentially, making it computationally expensive to search through all possibilities.
- **Complexity**: Some games have complex rules and branching factors, making it difficult for search algorithms to explore all potential moves within a reasonable time frame.
- **Memory Requirements**: Storing all game states and moves in memory can be impractical for games with large state spaces, leading to memory limitations.
- **Optimality**: Some search algorithms may not always find the optimal solution or may require significant computational resources to do so.

- **Adversarial Nature**: In games with an adversarial component, opponents may make suboptimal moves, leading to challenges in predicting their behavior accurately.
- **Heuristic Accuracy**: The effectiveness of heuristic evaluation functions can vary, impacting the performance of search algorithms, especially in games with complex strategies.
- **Real-time Constraints**: For real-time games, search algorithms must make decisions within strict time limits, which can restrict the depth of the search and the quality of the decisions made.
- **Incorporating Uncertainty**: Games with elements of uncertainty or hidden information may pose challenges for traditional search algorithms that assume complete knowledge of the game state.

Constraint satisfaction problems (CSPs)

- A **Constraint Satisfaction Problem** is a mathematical problem where the solution must meet a number of constraints.
- In a CSP, the objective is to assign values to variables such that all the constraints are satisfied.
- CSPs are used extensively in Artificial intelligence for decision-making problems where resources must be managed or arranged within strict guidelines.

Constraint satisfaction problems (CSPs)

- A Constraint Satisfaction Problem is a mathematical problem where the objective is to assign values to variables such that all the constraints are satisfied.
  Many AI applications use CSPs to solve decision-making problems. Common applications of CSPs include:
- **Scheduling:** It assigns resources like employees or equipment while respecting time and availability constraints.
- **Planning:** Organize tasks with specific deadlines or sequences.
- **Resource Allocation:** Distributing resources efficiently without overuse

Components

1. Variables: Things we need to find values for. For example, in a Sudoku puzzle each empty cell is a variable that needs a number.

2. Domains: The set of possible values that a variable can have. In Sudoku the domain for each cell is the numbers 1 to 9.

3. Constraints: The rules that restrict how variables can be assigned values. There are different types of constraints:

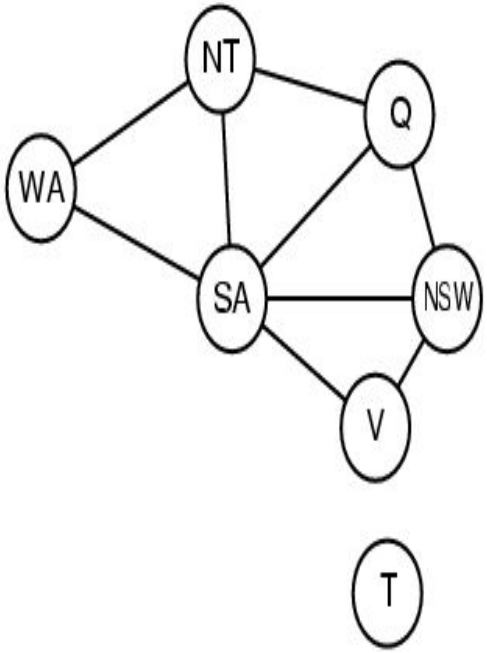
Unary: Apply to a single variable like "this cell cannot be 5".

Binary: Involve two variables like "these two cells cannot have the same number".

Higher-order: Involve three or more variables like "each row in Sudoku must have all numbers from 1 to 9 without repetition".

Constraint satisfaction problems (CSPs)

- Varieties of constraints:
  - Unary constraints involve a single variable,
    - e.g., SA ≠ green
  - Binary constraints involve pairs of variables,
    - e.g., SA ≠ WA
  - Higher-order constraints involve 3 or more variables,
    - e.g., cryptarithmetic column constraints



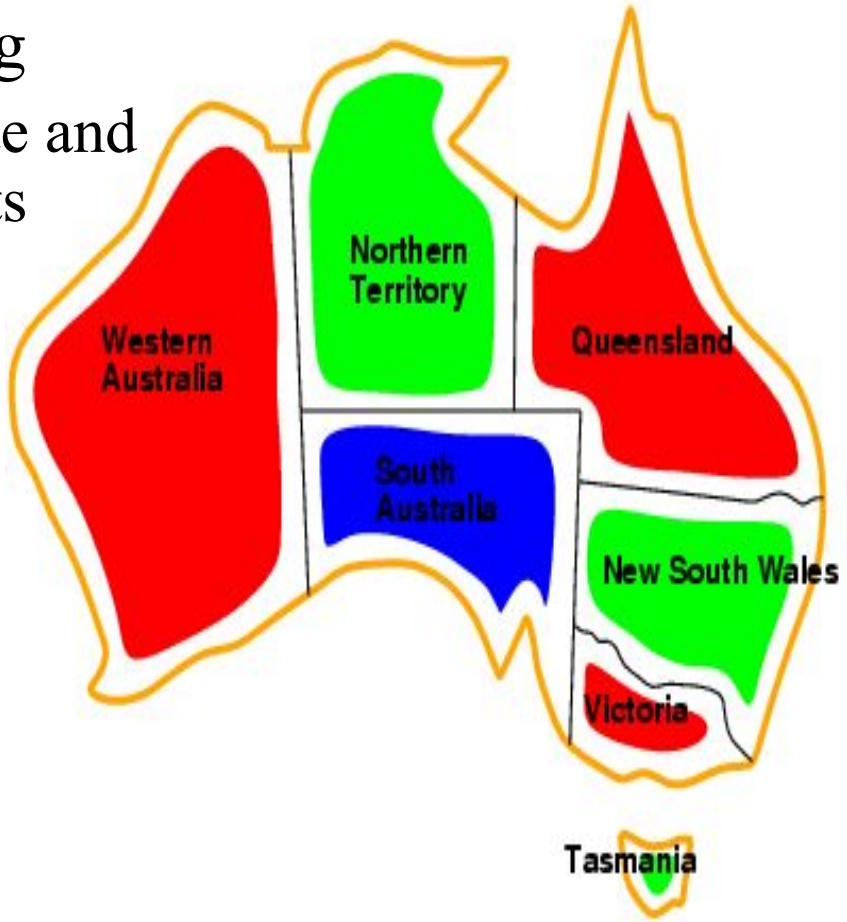
Constraint satisfaction problems (CSPs)

- Example: Map-Coloring
  - **Variables** WA, NT, Q, NSW, V, SA, T
  - **Domains** Di = {red, green, blue}
  - **Constraints**: adjacent regions must have different colors
    - e.g., WA ≠ NT, or (WA,NT) in {(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)}



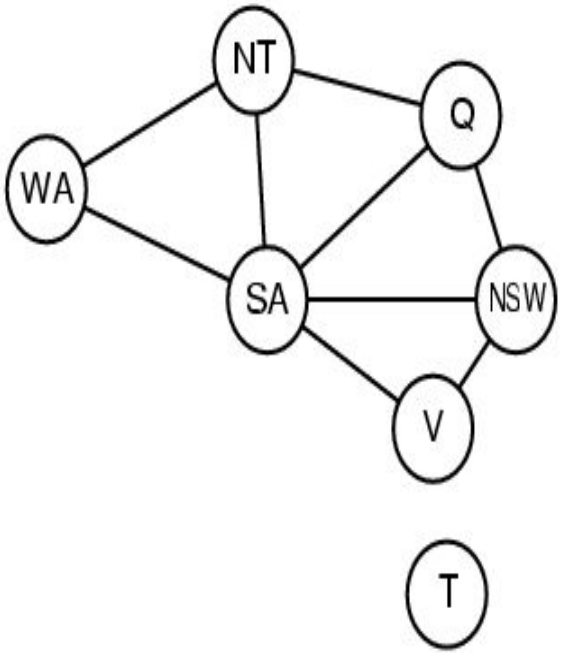
Constraint satisfaction problems (CSPs)

- Example: Map-Coloring
  - Solutions are complete and consistent assignments
  - e.g.,
    - WA = red,
    - NT = green,
    - Q = red,
    - NSW = green,
    - V = red,
    - SA = blue,
    - T = green



Constraint satisfaction problems (CSPs)

- Constraint graph:
  - The map-coloring problem represented as a constraint graph
  - Constraint graph: nodes are variables, arcs are constraints
  - In Binary CSP: each constraint relates two variables



Real-world Constraint Satisfaction Problems (CSP):

Scheduling: A fundamental CSP problem is how to efficiently and effectively **schedule resources like personnel, equipment, and facilities**. The constraints in this domain specify the availability and capacity of each resource, whereas the variables indicate the time slots or resources.

Vehicle routing: Another example of a CSP problem is the issue of **minimizing travel time or distance by optimizing a fleet of vehicles' routes**. In this domain, the constraints specify each vehicle's capacity, delivery locations, and time windows, while the variables indicate the routes taken by the vehicles.

Assignment: Another typical CSP issue is how to **optimally assign assignments or jobs to humans or machines.** In this field, the variables stand in for the tasks, while the constraints specify the knowledge, capacity, and workload of each person or machine.

Sudoku: The well-known puzzle game Sudoku can be modeled as a CSP problem, where the variables stand in for the grid's cells and the constraints specify the game's rules, such as prohibiting the repetition of the same number in a row, column, or area.

Constraint-based image segmentation: The segmentation of an image into areas with various qualities (such as **color, texture, or shape**) can be treated as a CSP issue in computer vision, where the variables represent the regions and the constraints specify how similar or unlike neighboring regions are to one another.

Inference in CSPs and

Inference in CSPs

- **Inference** means deriving new information from the current assignments to **reduce the possible values (domains)** of variables.
- **Main purpose:**
  - Reduce the search space.
  - Detect failure early.
  - Remove values that cannot lead to a solution.

- **Inference in a Constraint Satisfaction Problem (CSP)** means using the constraints and the values already assigned to variables to **deduce which values are no longer possible**.
- In simple words:
  - **Inference = Remove impossible values before searching further.**
- This helps reduce the **search space** and allows us to detect failure early.

Common inference techniques:

1. **Node Consistency** – removes values violating unary constraints.
2. **Arc Consistency** – removes values that have no supporting value in a neighboring variable.
3. **Forward Checking** – after assigning a variable, removes inconsistent values from neighboring variables.
4. **Maintaining Arc Consistency (MAC)** – maintains arc consistency after every assignment.

- **Example:**
  If WA = Red in the Australia map-coloring problem, neighboring regions such as NT and SA cannot be Red.

Node Consistency

- **Node consistency** deals with **unary constraints**.

  A unary constraint involves only one variable.

- **Example**

  Suppose:

  X = {1,2,3,4,5}

  Constraint:

  X > 2

  Remove values that violate the constraint:

  Before: X = {1,2,3,4,5} After: X = {3,4,5}

  So:

  X = {1,2,3,4,5} ↓ X > 2 ↓ X = {3,4,5}

- **Purpose**

  Remove values that violate the variable's own constraint.

Arc Consistency

Arc consistency is one of the most important inference techniques in CSPs.

- Consider two variables:

  X -------- Y

  Suppose:

  X = {1,2,3} Y = {2,3}

  Constraint:

  X<Y We check each value of X.

- **Check X = 1**

  Is there a value in Y greater than 1?

  Yes:

  Y = 2 or 3

  So **1 remains**.

Check X = 2

Is there a value in Y greater than 2?

Yes:

Y = 3

So **2 remains**.

Check X = 3

Is there a value in Y greater than 3?

No.

Therefore:

3 is removed.

Result:

Before: X = {1,2,3} Y = {2,3} X < Y After:
X = {1,2} Y = {2,3}

This is **arc consistency**.

Important definition

A variable value is **arc consistent** if there is at least one compatible value in the neighboring variable's domain.

Forward Checking

Forward checking is performed **after assigning a value to a variable**.

- Consider:

  A ---- B ---- C

- Domains:

  A = {R,G,B} B = {R,G,B} C = {R,G,B}

- Constraint:

  Connected regions must have different colors.

- Suppose:

  A = Red

- Forward checking looks at the **unassigned neighbor B**.

Since:

$B \neq A$

and:

A = Red

we remove Red from B:

B = {G,B}

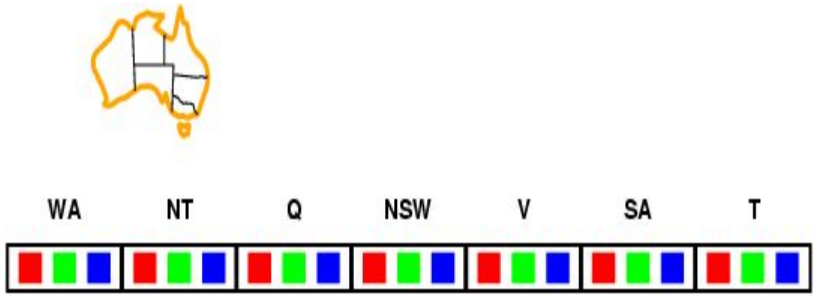
This prevents the algorithm from later trying:

A = Red B = Red

which is already known to be invalid.

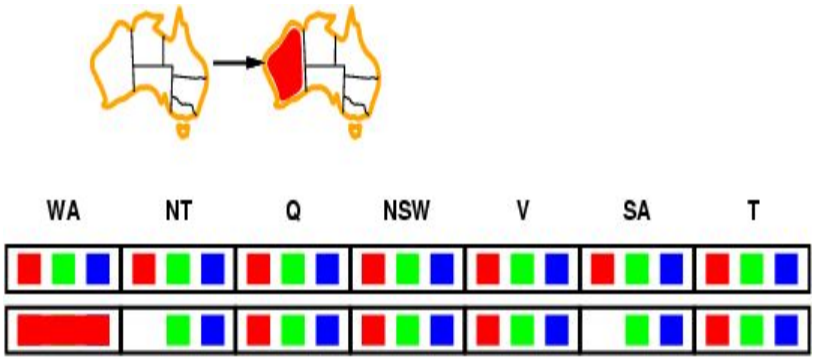
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



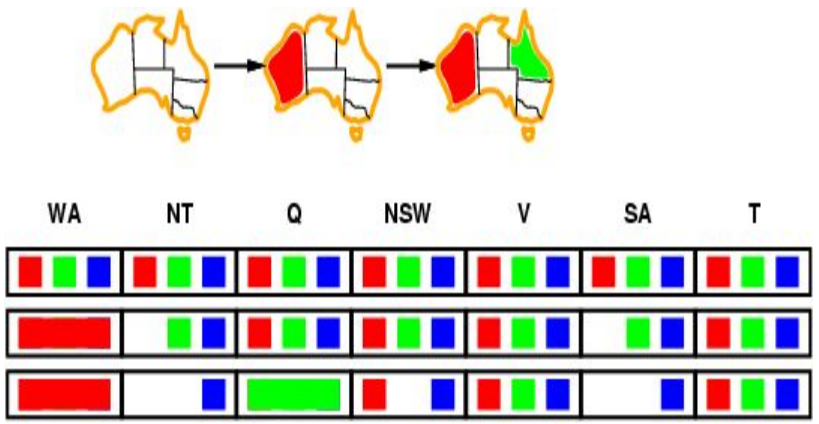
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



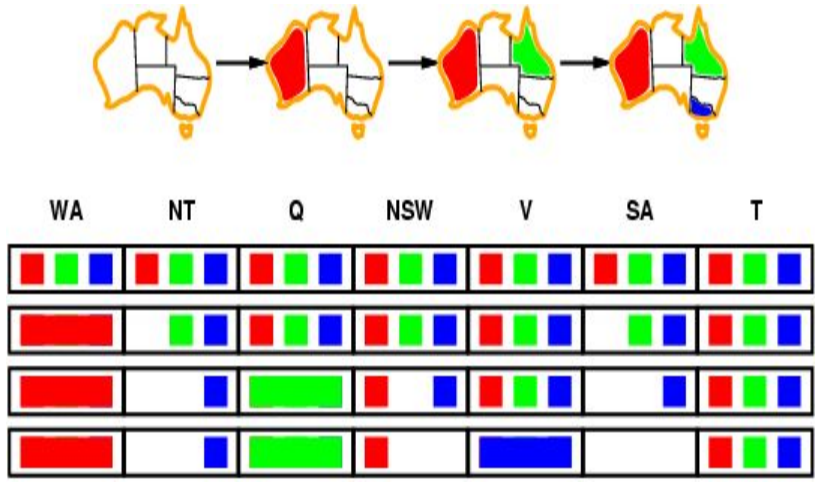
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



Path Consistency

- Path consistency involves **three variables**.
- Path consistency checks whether an assignment to two variables can be extended through a **third variable**.
- Suppose:

  $X = \{1,2\}$ $Y = \{1,2\}$ $Z = \{1\}$
- Constraints:

  $X \neq Z$ $Y \neq Z$
- Since:

  $Z = 1$
- we know:

  $X \neq 1$ $Y \neq 1$
- Therefore:

  $X = \{2\}$ $Y = \{2\}$
- But if there is also a constraint:

  $X \neq Y$
- then:

  $X = 2$ $Y = 2$

  violates the constraint.
- Therefore, the combination has no valid solution.

Maintaining Arc Consistency (MAC)

- **MAC = Maintaining Arc Consistency**

  MAC is a stronger technique used **during backtracking search**.

  After every assignment, it applies arc consistency to the CSP again.

- **Example**

  A -------- B -------- C

- Initially:

  A = {R,G,B} B = {R,G,B} C = {R,G,B}

- Assign:

  A = R

- MAC applies arc consistency:

  A = R B ≠ R B = {G,B}

- Now the change in B may affect C.

- **Important techniques in CSP**

Node Consistency → Arc Consistency → Forward Checking → Path Consistency → Maintaining Arc Consistency (MAC)

Backtracking search for CSPs

The **backtracking algorithm** is a **depth-first search** method used to systematically explore possible solutions in CSPs. It operates by assigning values to variables and backtracks if any assignment violates a constraint
Simple idea: Try → Check → Continue → If failure, Undo → Try another value.

How it works:

- The algorithm selects a variable and assigns it a value.
- It recursively assigns values to subsequent variables.
- If a conflict arises (i.e., a variable cannot be assigned a valid value), the algorithm backtracks to the previous variable and tries a different value.
- The process continues until either a valid solution is found or all possibilities have been exhausted.

This method is widely used due to its simplicity but can be inefficient for large problems with many variables.

Backtracking search

- Variable assignments are commutative, i.e.,

  [WA = red then NT = green]

  same as [NT = green then WA = red]

- Only need to consider assignments to a single variable at each node
- Depth-first search for CSPs with single-variable assignments is called backtracking search

Backtracking is a **depth-first search** method used to find a CSP solution.

- **Basic steps**

1. Select an unassigned variable.
2. Select a possible value.
3. Check whether the assignment is consistent.
4. If consistent → continue with the next variable.
5. If inconsistent → try another value.
6. If no value works → **backtrack** to the previous variable.
7. Continue until all variables are assigned.

Backtracking search

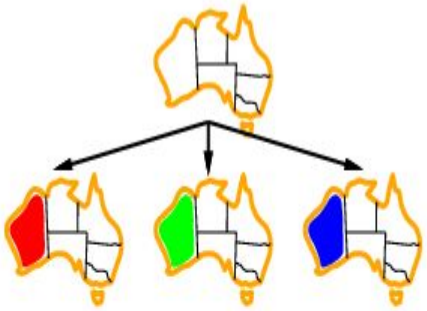
```
function BACKTRACKING-SEARCH( csp) returns a solution, or failure
    return RECURSIVE-BACKTRACKING({}, csp)

function RECURSIVE-BACKTRACKING( assignment,csp) returns a solution, or
failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE( Variables[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment according to Constraints[csp] then
            add { var = value } to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failue then return result
            remove { var = value } from assignment
    return failure
```

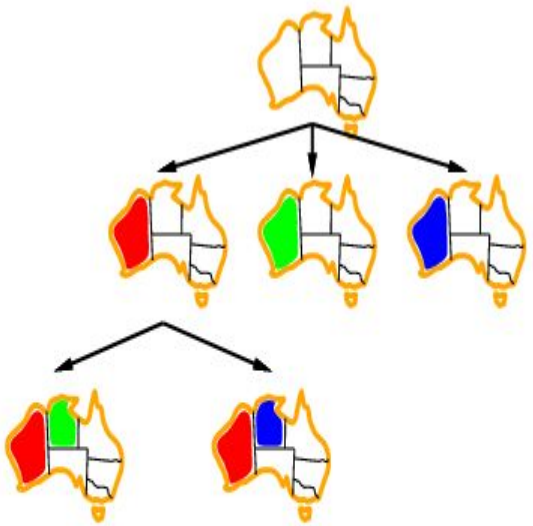
Backtracking example



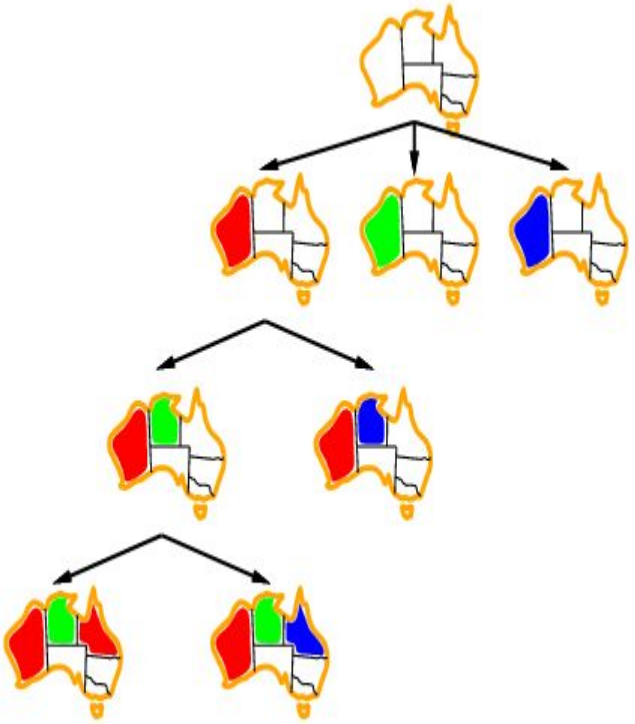
Backtracking example



Backtracking example



Backtracking example



Most constrained variable

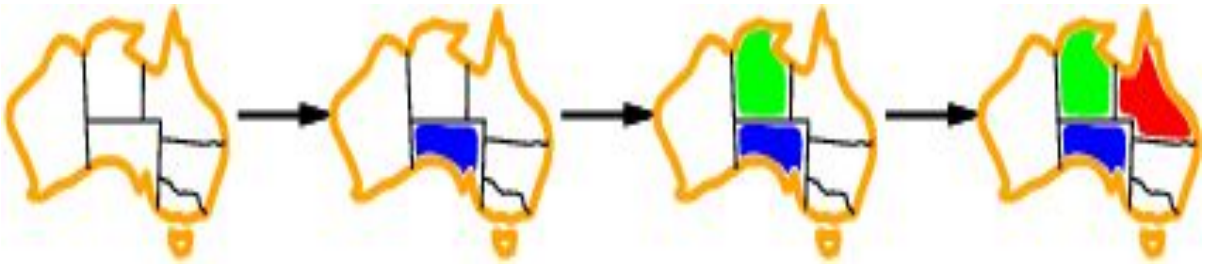
- Most constrained variable:
  choose the variable with the fewest legal values



- minimum remaining values (MRV) heuristic

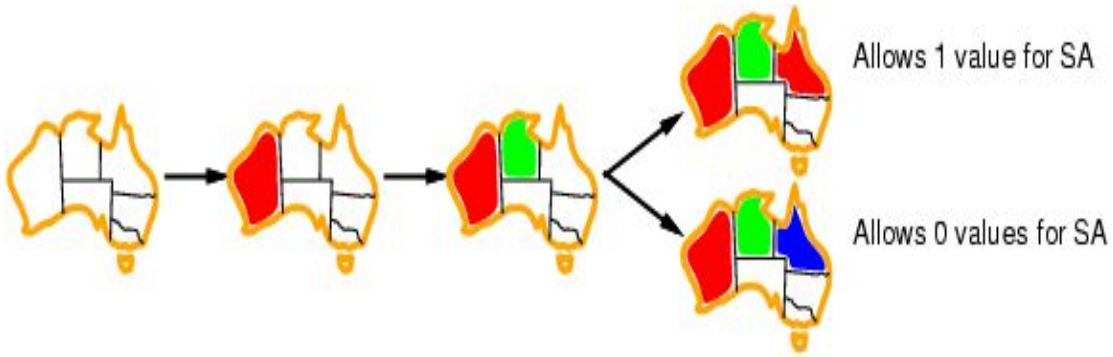
Most constraining variable

- A good idea is to use it as a tie-breaker among most constrained variables
- Most constraining variable:
  - choose the variable with the most constraints on remaining variables



Least constraining value

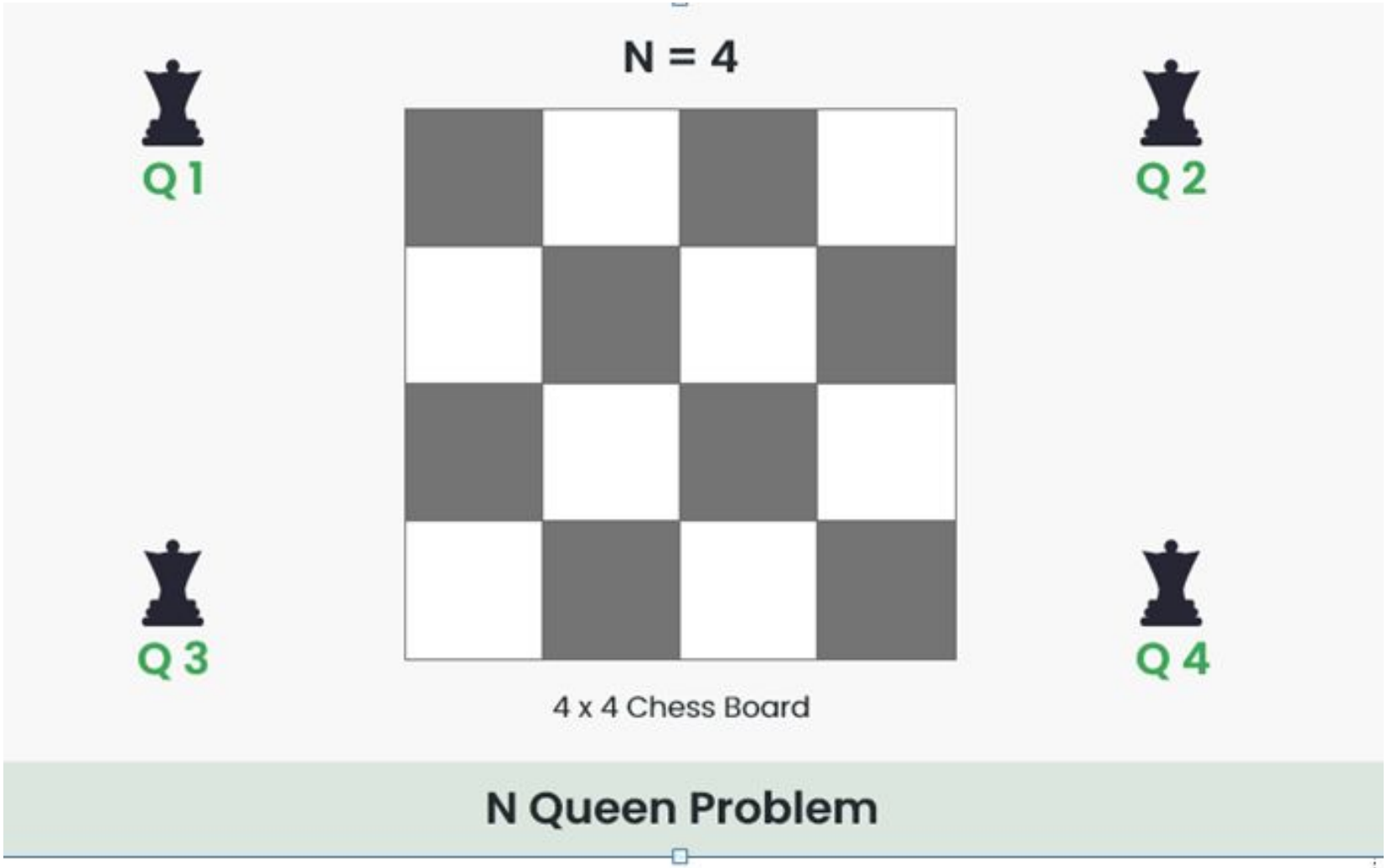
- Given a variable to assign, choose the least constraining value:
  - the one that rules out the fewest values in the remaining variables
- Combining these heuristics makes 1000 queens feasible



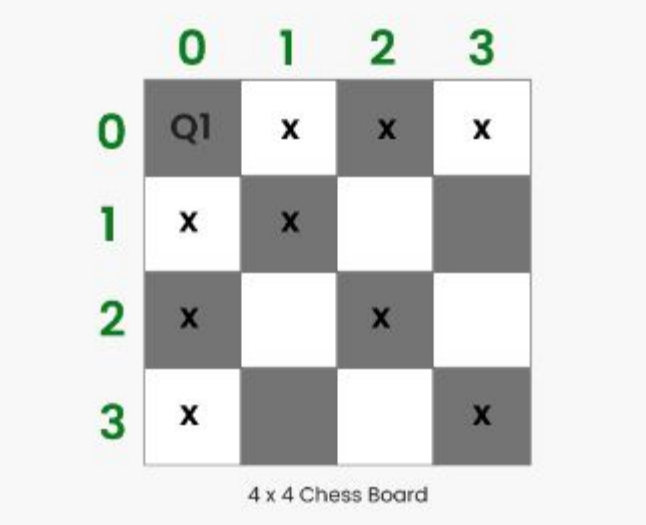
Backtracking in the 4-Queens Problem

- The **4-Queens problem** is one of the best examples for understanding backtracking.

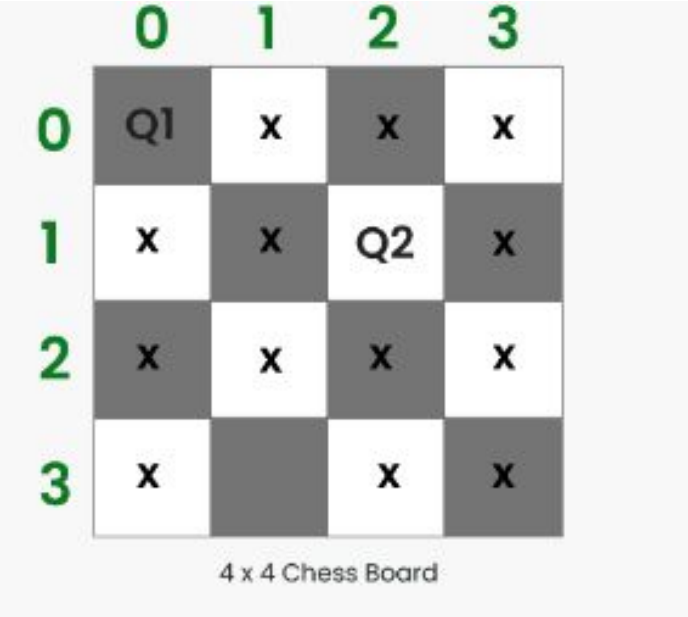
- **Problem**
  - Place 4 queens on a 4 × 4 chessboard such that:
  - No two queens are in the same row.
  - No two queens are in the same column.
  - No two queens are on the same diagonal.
  - We place **one queen in each row**.


N = 4
Q 1
Q 2
Q 3
Q 4
4 x 4 Chess Board
N Queen Problem

- **Step 1:**
- Put our first Queen (**Q1**) in the **(0,0)** cell .
- '**x**' represents the cells which is not safe i.e. they are under attack by the Queen (**Q1**).
- After this move to the next row [0 -> 1].



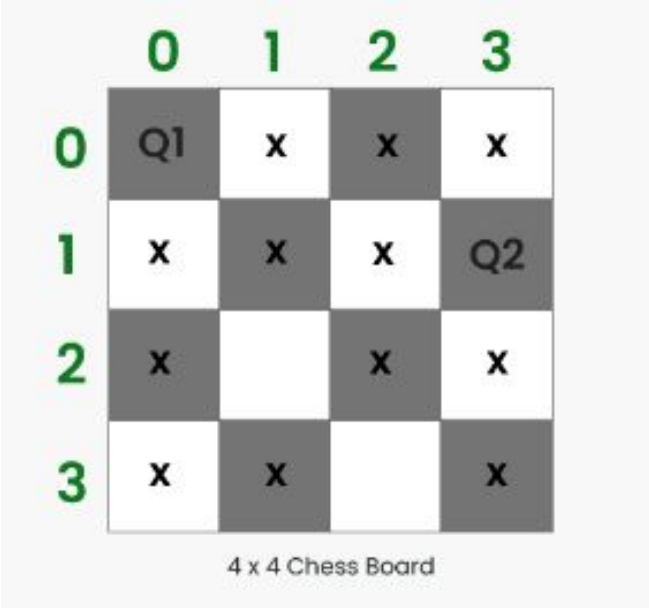
- **Step 2:**
- Put our next Queen (**Q2**) in the **(1,2)** cell .
- After this move to the next row [1 -> 2].



Step 3:

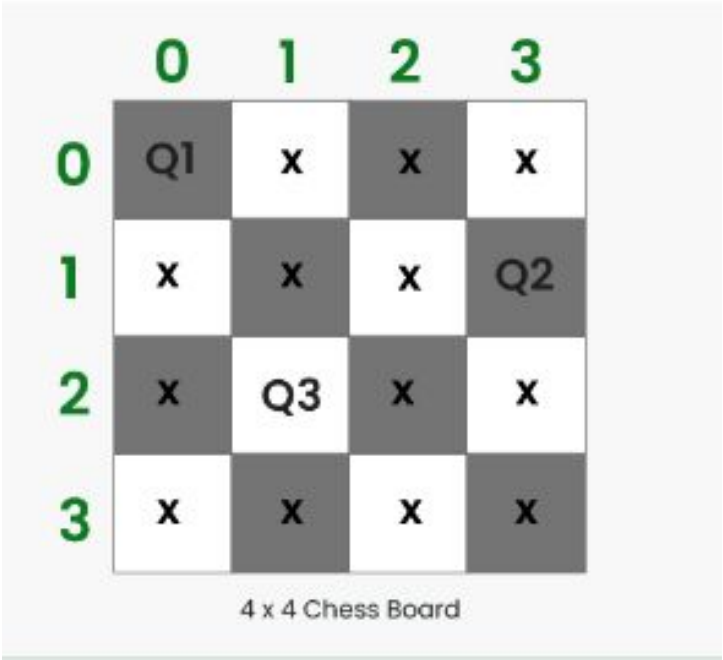
- At row 2 there is no cell which are safe to place Queen (Q3) .
- So, backtrack and remove queen Q2 queen from cell (1, 2) .

- **Step 4:**
- There is still a safe cell in the row 1 i.e. cell (1, 3).
- Put Queen (**Q2**) at cell (1, 3).



Step 5:

- Put queen **(Q3)** at cell (2, 1).

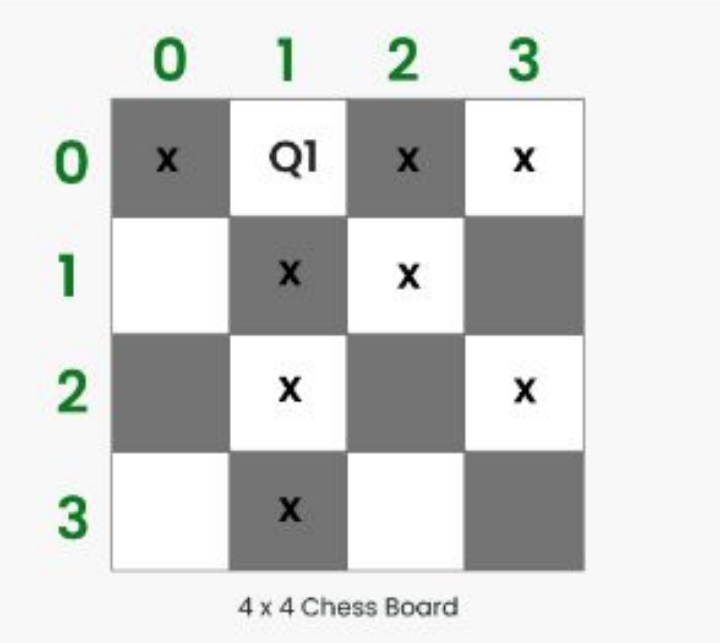


Step 6:

1. There is no any cell to place Queen (Q4) at row 3.
2. Backtrack and remove Queen (Q3) from row 2.
3. Again there is no other safe cell in row 2, So backtrack again and remove queen (Q2) from row 1.
4. Queen (Q1) will be remove from cell (0,0) and move to next safe cell i.e. (0 , 1).

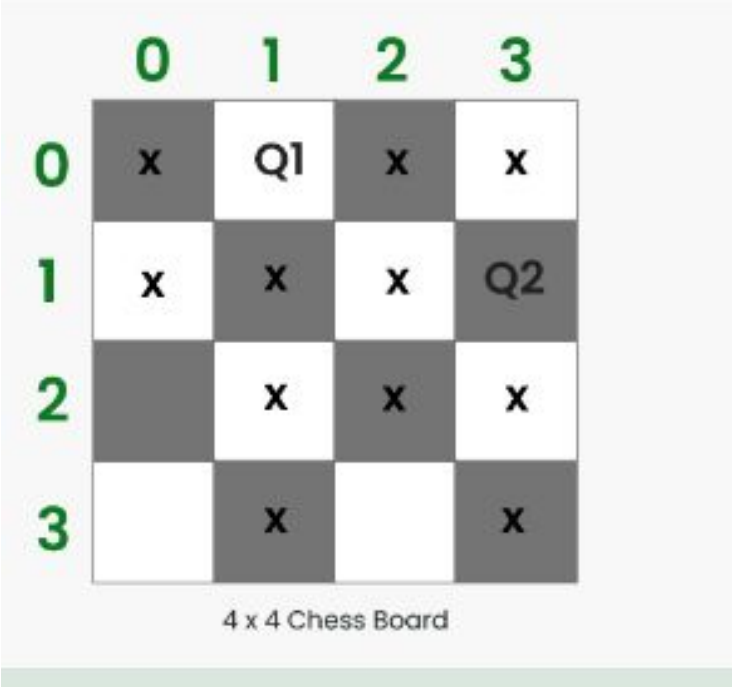
Step 7:

- Place Queen Q1 at cell (0 , 1), and move to next row.



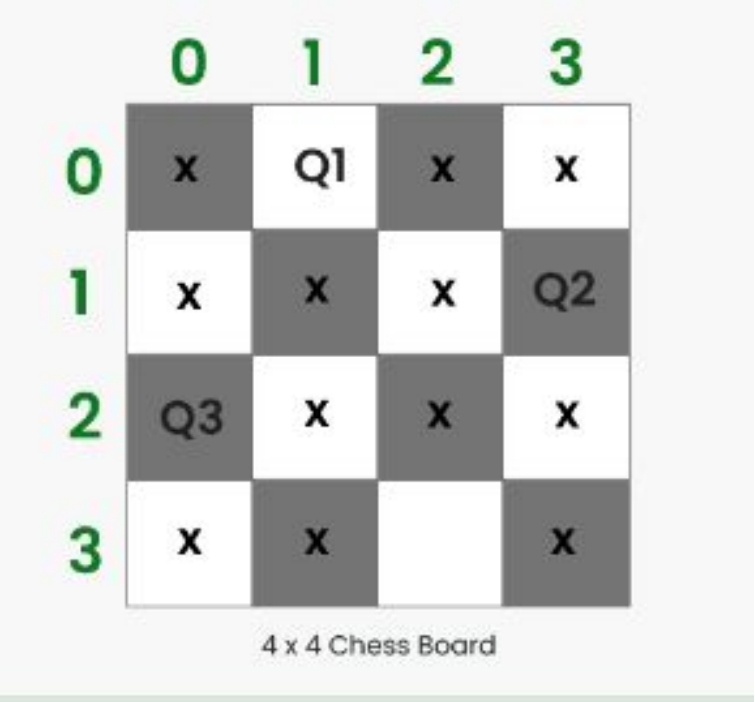
Step 8:

- Place Queen **Q2** at cell **(1 , 3)**, and move to next row.

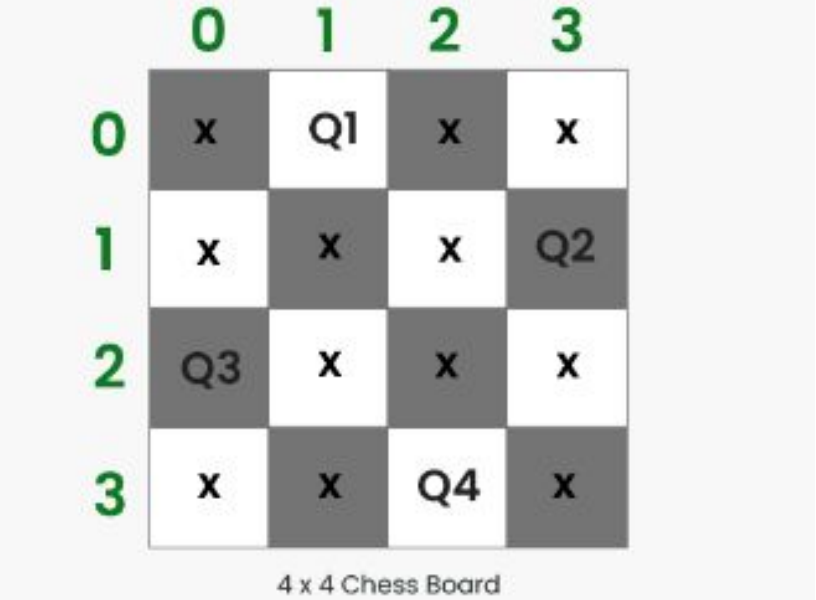


Step 9:

- Place Queen **Q3** at cell **(2 , 0)**, and move to next row.



- **Step 10:**
- Place Queen **Q4** at cell **(3 , 2)**, and move to next row.
- This is one possible configuration of solution



Follow the steps below to implement the idea:

1. Make a recursive function that takes the state of the board and the current row number as its parameter.
2. Start in the topmost row.
3. If all queens are placed, return **true**
4. For every row.
    1. Do the following for each column in current row.
        1. If the queen can be placed safely in this column
            1. Then mark this **[row, column]** as part of the solution and recursively check if placing queen here leads to a solution.
        2. If placing the queen in **[row, column]** leads to a solution, return **true**.
        3. If placing queen doesn't lead to a solution then unmark this **[row, column]** and track back and try other columns.
5. If all columns have been tried and nothing worked, return **false** to trigger backtracking.

Challenges

- **Scalability:** Large numbers of variables and constraints can create a very large search space.
- **Dynamic Constraints:** Real-world problems may change over time, requiring the solution to be updated.
- **No Feasible Solution:** Strict or conflicting constraints may make it impossible to find a valid assignment.

Reference

- **Text Books:**
  - Stuart Russell and Peter Norvig, “Artificial Intelligence: A Modern Approach”, Third edition, Pearson, 2003, ISBN :10: 0136042597

Thank You

Case Study:

- AlphaGo – AI in Strategic Board Games,
- Strategic Decision-Making in Imperfect Information Games- Libratus,
- Adversarial Search and Constraint Reasoning in Computer Chess – IBM deep blue

AI in Strategic Games: AlphaGo and Libratus

- These two systems are important examples of **AI for strategic decision-making**:
  AlphaGo → Strategic board game with **perfect information**
  Libratus → Strategic card game with **imperfect information**

- **AlphaGo** is an AI system developed by **DeepMind** to play the board game **Go**.

It became famous after defeating professional Go player **Lee Sedol** in a five-game match in 2016.

1. Why Go is difficult for AI?

1. Go has:
2. A very large number of possible moves.
3. A huge search space.
4. Long-term strategic consequences.
5. No randomness or hidden information.
6. Although the entire board is visible to both players, the number of possible game positions is enormous.

2. How AlphaGo Makes Decisions

AlphaGo combines:

7. **Neural Networks**
8. **Monte Carlo Tree Search (MCTS)**
9. **Self-play**
10. **Reinforcement Learning**

3. Policy Network

1. The **policy network** predicts promising moves.
2. For example:
3. Possible moves: A → 10% B → 60% C → 5% D → 25%
4. AlphaGo gives more attention to moves that appear promising.
5. **Purpose**
6. **Policy network = Which move should I consider?**

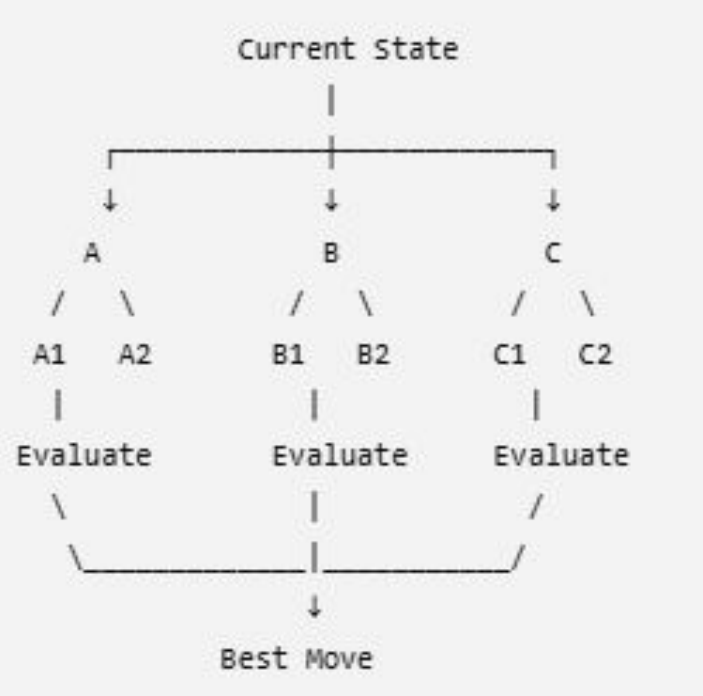
4. Value Network

7. The **value network** estimates the probability of winning from a particular board position.
8. For example:
9. Current position Probability of winning: Player A → 70% Player B → 30%
10. **Purpose**

Value network = How good is this position?

5. Monte Carlo Tree Search

- AlphaGo uses **Monte Carlo Tree Search (MCTS)** to explore possible future moves.
- Simple example:
- Current State



- Instead of exhaustively searching every possible move, AlphaGo focuses on promising branches.

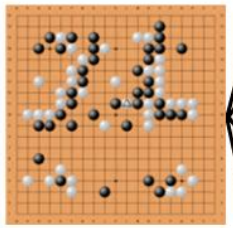
6. Why AlphaGo Was Important

1. AlphaGo demonstrated that AI could learn sophisticated strategies in a highly complex game.
2. It showed that:

Deep learning + search + reinforcement learning can produce powerful strategic decision-making.

AlphaGo Overview

based on: Silver, D. et al. Nature Vol 529, 2016
copyright: Bob van den Hoek, 2016



Input Board Position
as 19 x 19 Image
48 Feature Planes

Policy Network

Expert Games
130 000 Games
30 M Positions

Supervised Learning
SL Policy
Position --> Next Move
Accuracy: 56%

Fast Policy Network

Expert Games
140 000 Patterns
130 000 Games
30 M Positions

Supervised Learning
Fast Policy
Pattern --> Next Move
Accuracy: 24%

Reinforcement Learning Policy Network

Self-Play Games
1.3 M Games by
various versions
of RL Policy

Reinforcement Learning
RL Policy
Position --> Next Move
Wins 80% vs. SL Policy

Value Network

Self-Play Games
30 M Positions
by fixed version
of RL Policy

Reinforcement Learning
Position --> Win Probability
15 000 times faster than
MCTS Rollouts evaluations

AlphaGo

Monte Carlo Tree Search